Literature Review

Real-time Scheduling for Dynamic Coalitions of Heterogeneous Robots

Authored by

Jakob Bichler

Primary Supervisor:	Andreu Matoses Gimenez (PhD)
Supervising Professor:	Javier Alonso-Mora (Associate Professor)
Faculty:	Mechanical Engineering, TU Delft
Study Programm:	MSc Robotics
Lab:	Autonomous Multi-Robots Laboratory

Contents

Nomenclature

ACO . . . . . Ant Colony Optimization
CD . . . . . Complex Dependencies
CVaR . . . . . Conditional Value at Risk
DAG . . . . . Directed Acyclic Graph
EDF . . . . . Earliest Deadline First
FJSSP . . . . . Flexible Job-Shop Scheduling Problem
GA . . . . . Genetic Algorithm
GAN . . . . . Graph Attention Network
GATN . . . . . Graph Attention Transformer Network
GCAPCN . . . . . Graph Capsule Convolutional Network
GNN . . . . . Graph Neural Network
HFBS . . . . . Hybrid Filtered Beam Search
IA . . . . . Instantaneous Assignment
ID . . . . . In-Schedule Dependencies
IL . . . . . Imitation Learning
ILP . . . . . Integer Linear Programming
JSSP . . . . . Job Shop Scheduling Problem
LLM . . . . . Large Language Model
LSTM . . . . . Long short-term memory
MDP . . . . . Markov Decision Process
MILP . . . . . Mixed-Integer Linear Programming
MIP . . . . . MIxed Integer Programming
MLP . . . . . Multilayer Perceptron
MR . . . . . Multi-Robot Tasks
MRTA . . . . . Multi-Robot Task Assignment
MT . . . . . Multi-Task Robots
ND . . . . . No Dependencies
POMDP . . . . . Partially Ovservable Markov Decision Process
PSO . . . . . Particle Swarm Optimization
RL . . . . . Reinforcement Learning
SR . . . . . Single-Robot Tasks
ST . . . . . Single-Task Robots
STN . . . . . Simple Temporal Network
TA . . . . . Time-Extended Assignment
TSP . . . . . Traveling Salesman Problem
UAV . . . . . Unmanned Aerial Vehicle
UGV . . . . . Unmanned Ground Vehicle
USV . . . . . Unmanned Surface Vehicle
VRP . . . . . Vehicle Routing Problem
XD . . . . . Cross-Schedule Dependencies

1 Introduction

1.1. Preliminaries

The field of Scheduling and Multi-Robot Task Assignment includes many specialized approaches, and terms are often used differently depending on the context and paper, which might lead to confusion. This section defines the key terms used throughout the review to provide a consistent foundation.

Multi-Robot Task Assignment

Multi-robot task assignment (*MRTA*) aims to assign tasks to robots while meeting constraints to minimize an application-specific cost function [1]. The result is a (near-) optimal mapping from robots to tasks with corresponding starting times [2], called a schedule. Robots are assigned to tasks based on their required skills, with tasks varying in type and requirements. The value of completing each task can differ based on its finish time, the assigned robot, or other factors. Furthermore, task constraints influence the optimal schedule, since a specific order for task completion may be necessary [3], see Section 3.4 for more detail.

MRTA Taxonomy

In *MRTA*, effectively scheduling and assigning tasks requires a good understanding of the relationships between agents and tasks. In the following sections of this literature survey, the iTax taxonomy [4] will be used to systematically categorize the methods under review. Gerkey and Matarić [5] first introduced a foundational taxonomy for *MRTA* problems based on three key axes, shown in Table 1.1. Building upon this framework, the iTax taxonomy was proposed [4], which introduces an additional dimension that captures the degree of interdependence of agent–task utilities — a key factor influencing the complexity of *MRTA* problems. iTax categorizes *MRTA* task dependencies into four classes, also see Table 1.2.

Axis	**Description**
Single-Task Robots (*ST*) vs. Multi-Task Robots (*MT*)	Robots are capable of executing at most one task at a time (*ST*) or can handle multiple tasks simultaneously (*MT*).
Single-Robot Tasks (*SR*) vs. Multi-Robot Tasks (*MR*)	Tasks require exactly one robot to accomplish them (*SR*) or necessitate the cooperation of multiple robots (*MR*).
Instantaneous Assignment (*IA*) vs. Time-Extended Assignment (*TA*)	*MRTA* without considering future tasks (*IA*) or scheduling over a time horizon with knowledge of upcoming tasks (*TA*).

Table 1.1: Classification of *MRTA* problems according to [5]

Dependency Type	Description
No Dependencies (*ND*)	Tasks have independent utilities; the utility of assigning a task to an agent is unaffected by other tasks or agents.
In-Schedule Dependencies (*ID*)	The utility of a task for an agent depends on other tasks assigned to the same agent, introducing intra-agent scheduling considerations.
Cross-Schedule Dependencies (*XD*)	An agent's utility for a task depends on the tasks assigned to other agents, involving inter-agent coordination constraints.
Complex Dependencies (*CD*)	Utilities are affected by both inter-agent schedules and complex task decompositions, requiring simultaneous consideration of task allocation and decomposition.

Table 1.2: Dependency Types in *MRTA* according to [4]

Farinelli et al. provide another taxonomy for classifying *MRTA* problems in [6] that focuses on coordination within the robot team. They distinguish on different dimensions - Knowledge level (unaware vs. aware of other robots states), Coordination level (strong vs. weak coordination protocol), Organizational level (centralized vs. decentralized). Further differentiations include the team composition (homogeneous vs. heterogeneous robots), team size and whether the communication is direct or indirect.

Heterogeneous Robots

A team of robots is considered heterogeneous if the members offer different capabilities [6]. According to [7] robots can either differ in physical or behavioral aspects. Physical heterogeneity refers to differences in hardware or physical constraints. For example, a team consisting of a fast drone equipped with a camera and a slow mobile manipulator that can lift heavier objects. Behavioral heterogeneity occurs when the software that produces the behavioral model differs. Some robots might try to maximize a different objective function than others.

For this work, a heterogeneous robot team is defined as physically different robots that work together to optimize for a shared goal.

Dynamic Coalition Formation

In MRTA robots might need to form coalitions (also called sub-teams) to execute a task when no single robot can provide all required capabilities [8]. Coalitions can either be formed statically or dynamically. Static formations are grouped once and remain the same over the complete task assignment horizon. Dynamic coalitions change their formation from task to task to ensure efficiency [9]. The requirement to form coalitions is closely related to tackling multi-robot (*MR*) tasks in the iTax taxonomy [4].

1.2. Motivation

Autonomous Multi-robot systems have gained increasing recognition over the last years [10] with a focus on problems involving single-task robots and single-robot tasks (*ST-SR*), as can be seen in Figure 1.1. Autonomous Multi-robot systems are designed for complex, real-world settings, including earthquake disaster response scenarios [11], autonomous construction [12], production assembly processes [13], or search and rescue missions [14]. The rising prominence of these systems makes efficient scheduling and coordination essential to optimize resource usage and minimize operational time [15]. A coordinated robotic team develops synergies among teammates, achieving performance that exceeds the sum of individual robot capabilities. Such teams improve performance and enhance system robustness as a multi-robot team is more resilient against individual robot failures and performance bottlenecks [16, 17]. Dynamic coalition formation enables teams to accomplish complex tasks that are infeasible for a single robot [17–19]. *MRTA* approaches incorporating dynamic coalition formation are solving *MR* tasks. However, as shown in Figure 1.1, these *MR* problems have historically received less attention compared to their *SR* counterparts, though interest in them has increased in recent years.

In practice, relying on a team of homogeneous, interchangeable robots where each robot possesses all skills can become impractical if task requirements are numerous, highly diverse or involve different sensors and actuators [20]. Furthermore, using heterogeneous robots with specialization and redundancy offers economic benefits. It allows leveraging existing specialized robots [21] or deploying smaller, simpler robots that are more cost-effective to implement and maintain [17, 22] and are more robust to failures [15]. A modular system enables the reuse of robots, reducing the need for a completely new fleet when adapting to changing use cases.

Multi-robot teams operate in dynamic environments where sudden changes, new tasks, unexpected task requirements, robot malfunctions, or moving obstacles can occur [10]. The ability to adapt and replan quickly is crucial to handle these challenges effectively. Efficient, real-time methods are needed to regenerate feasible solutions when conditions change to maintain optimal performance.

To further enhance applicability to real-world scenarios, precedence constraints can be modeled, which impose a logical sequence among tasks, ensuring some are completed before others begin, reflecting their underlying dependencies [23]. This is important for scenarios, where the completion of prior tasks determines whether a subsequent task can be performed. For example, in autonomous construction, clearing a site and gathering materials must precede the assembly phase [12].

Learning-based methods have gained increasing traction for *MRTA* problems requiring real-time decision-making and scalability in recent years. These models can encode task and robot relationships efficiently while leveraging *RL* or *IL* to optimize performance.

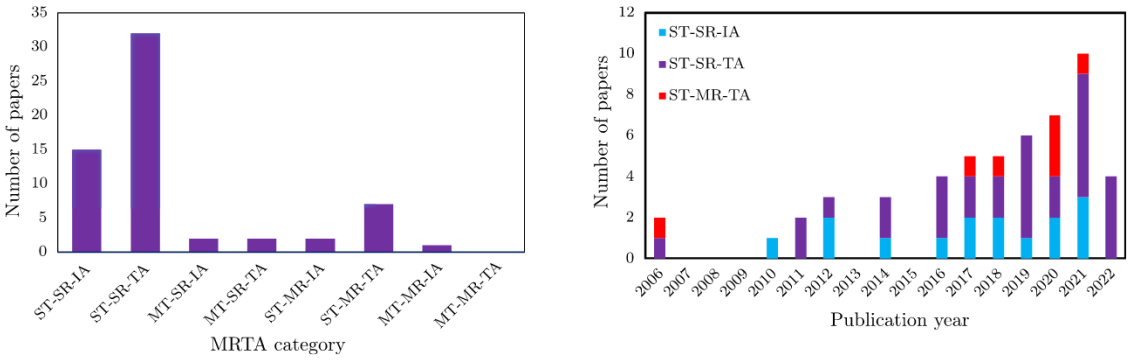

Figure 1.1: Distribution of papers on *MRTA* problems from 2006 to 2022 ([10]). Left: Number of papers per *MRTA* category. Right: Number of papers published per year, grouped by *MRTA* categories.

Given the importance of real-time methods for dynamic coalition formation with heterogeneous robots for real-world applications — and their limited attention in existing research — this thesis aims to advance research in this area.

1.3. Research Question

The thesis aims to answer the following research question, which will be refined as the project progresses:

Research Question

How can learning methods be designed to efficiently solve scheduling problems involving heterogeneous robots, dynamic coalitions, and precedence constraints in real-time?

1.4. Problem Definition

This section describes the use case the thesis will address.

We want to find (near-)optimal schedules in real-time for this heterogeneous team of robots while enabling dynamic coalition formation and respecting precedence constraints. We assume dynamically formed coalitions to solve tightly coupled *MR* tasks, requiring all assigned robots to be present and execute the task simultaneously rather than sequentially [24]. Real-time in this context refers to the ability to re-plan if the environment changes during the execution of the generated schedule, like unexpected tasks, requirements, disturbances, or human interaction. To do this efficiently, we are aiming to be able to generate new schedules with approximately 1 Hz on a consumer-grade laptop from 2020 with an AMD Ryzen 7 4800H CPU (8 cores, 2.9 GHz) and NVIDIA GeForce GTX 1650 (4 GB GDDR6 VRAM), as this is the laptop available to the author. The class of problems we are interested in is restricted to teams of 3-10 heterogeneous robots, each robot possessing one or more of 3-10 available skills, and coalitions can be formed with sizes of 1, 2, or 3 robots.

A motivating real-world example for this class of problems is a robotic bar environment, in which this team of robots has to take and prepare orders, serve customers, and clean. Tasks can appear during execution as new customers may arrive or new orders might be placed. Coalition formation might be required for tasks like unloading multiple dishes from a mobile base transport robot with a mobile manipulator. An example precedence constraint would be that the preparation of a drink has to precede delivering the drink. Early real-world examples show service robots being deployed in the hospitality industry [25], as well as robots being specifically designed to operate in bars and prepare drinks [26]. The Autonomous Multi-Robots Lab of the TU Delft [1] offers a variety of ground and flying robots and mobile manipulators that can be used for real-world testing.

This problem is an instance of the *ST-MR-TA-XD* group within the iTax taxonomy [4]. According to the framework by Farinelli et al. [6], the problem can be categorized as cooperative heterogeneous teams, where robots are aware of other robots' states and actions, and are strongly coordinated. Figure 1.2 shows an example of a mission plan and Figure 1.3 visualizes the corresponding schedule.

There are several assumptions to be made to scope the problem:

A1: There exists a perception/task decomposition system, that due to previously learned affordances [27] can generate tasks with according:

- (a) Locations
- (b) Estimated task durations (per robot)
- (c) Required skills
- (d) Precedence constraints relating to other tasks

A2: The robot team composition is known, including:

- (a) Number of robots
- (b) Position of each robot
- (c) Skills of each robot
- (d) Travel speeds

[1] https://autonomousrobots.nl

A3: The robots are equipped with a perception system, capable of:

(a) Locating obstacles for collision avoidance

(b) Assessing whether a task was successfully completed after attempted execution

A4: The robots are equipped with a low-level control/motion planning system, capable of:

(a) Generating feasible/collision-free trajectories for task execution

(b) Locally avoiding dynamic obstacles, e.g. other robots, customers

(c) Execute the generated trajectories, both for traveling and task execution

A5: The scheduling algorithm will be designed to run in real-time therefore enabling fast optimistic re-planning, so it can be assumed that:

(a) Each task succeeds every time, task failure triggers re-planning

(b) Robots have 100% uptime without malfunctions

(c) Robots battery constraints do not need to be modeled

A6: All robots are connected through a stable communication network so the schedules generated by a centralized planner can be sent and executed

Note that during the thesis, some of the assumptions might be eased to create a more realistic system or more assumptions might be added due to issues arising during implementation.

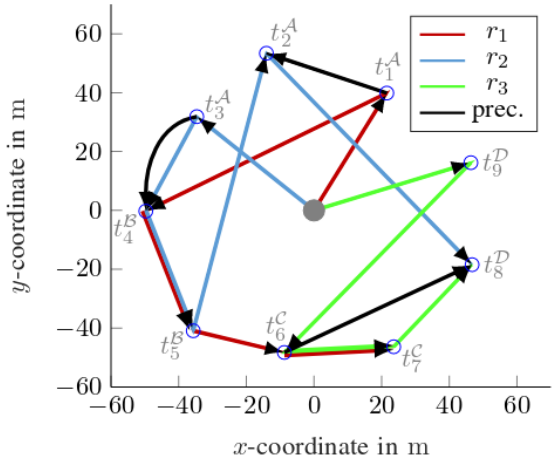

Figure 1.2: Example mission graph from [15] representing a solution for a problem with 9 tasks of type A,B,C,D (3A, 2B, 2C, 2D). Colored paths indicate each robot's schedule, while black edges represent precedence constraints. Tasks B and C require coalition formation.

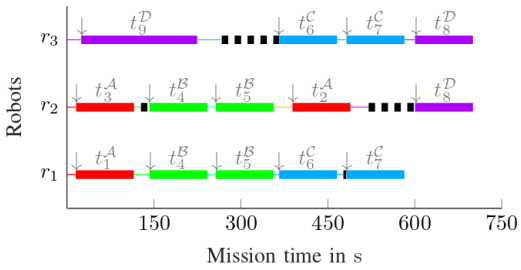

Figure 1.3: Gantt chart from [15] corresponding to the mission graph in Figure 1.2. Identically colored blocks represent tasks of the same type. Thin lines indicate travel times, while dashed black lines indicate waiting periods. Gray downward arrows mark the start time of each task.

1.5. Aims, Methodology and Structure

This section provides an overview of the objectives guiding the Literature Review, details the methodology used for identifying and selecting relevant studies, and explains the structure of the review.

Aims of Literature Review

This literature survey aims to answer the following literature review questions:

Literature Review Question 1

Which methods are suitable for scheduling heterogeneous teams of robots with dynamic coalition formation and precedence constraints in real-time?

- **Sub-Question 1A:** How are heterogeneity, dynamic coalition formation, and precedence constraints modeled in the literature?

Literature Review Question 2

How can methods for scheduling heterogeneous teams of robots with dynamic coalitions be objectively compared?

- **Sub-Question 2A:** Does a benchmark/dataset for scheduling heterogeneous robots with dynamic coalitions exist?
- **Sub-Question 2B:** Which factors complicate objective comparison?

Methodology

The review is guided by the use case described in Section 1.4 and only methods that could potentially (be extended to) solve it are considered and analyzed. Several relevant articles were initially reviewed to identify suitable search terms. The query derived from the research question is structured around four key components: ”robot”, ”coalition formation”, ”heterogeneous”, and ”scheduling”, each with corresponding synonyms. The search required terms related to ”robot” and ”scheduling”. However, to make the query more flexible, it was adjusted to include papers covering either ”heterogeneous” or ”coalition formation” individually:

Search Query

(“robot” **OR** ”agent”)
AND
(”scheduling” **OR** (task **AND** (allocation **OR** assignment **OR** planning)))
AND
(”coalition formation” **OR** ”coalition forming” **OR** ”subteaming” **OR** ”sub-teams” **OR** ”team formation” **OR** ”collaborative” **OR** ”heterogeneous” **OR** ”multi-skilled”)

This query was used in the databases Scopus [2], IEEE Xplore[3] and Google Scholar [4] in October, November, and December 2024.

The abstracts of the resulting papers were scanned for relevance, prioritizing more recent approaches under the assumption that state-of-the-art performance generally improves over time. When a promising paper was identified, both its references and subsequent citing papers were explored to expand the search. Due to a lack of diverse papers specifically covering heterogeneous *MRTA* with dynamic coalition formation, the final selection also included the most promising approaches for heterogeneous *MRTA* without coalition formation and homogeneous *MRTA* with coalition formation. These methods were included as they could potentially be expanded to address the use case described in Section 1.4. For this review, search terms related to precedence constraints and real-time considerations were excluded to avoid excessively narrowing the scope.

[2] https://www.scopus.com
[3] https://www.ieeexplore.ieee.org
[4] https://scholar.google.com/

Additional papers not analyzed in depth in Section 4 were read and used to support the discussion, provide context and offer complementary ideas and approaches.

Literature Review Structure

The following literature review examines common modeling assumptions in Section 3, comparing how heterogeneity, dynamic coalition formation, task constraints, and uncertainty are modeled in the literature. Section 4 presents a comprehensive analysis of 20 full solution methods found in the literature, that either address scheduling of heterogeneous robots (Section 4.1), dynamic coalition formation (Section 4.2), or both (Section 4.3). Each method's approach is explained and factors like heterogeneity, coalition formation, and real-time applicability are summarized in the overview table in Section 4.4. Next, Section 5 discusses the lack of standardized datasets and benchmarks, as well as the resulting difficulties in objectively comparing different algorithms due to the very diverse nature of use cases, assumptions, and reported metrics. Finally, Section 6 provides a concluding synthesis, answering the Literature Review Questions detailed in 1.5 and connects the findings with an outlook on promising future research directions.

2

Brief Overview of related Problems

In this section, we briefly outline two related scheduling formulations — the Vehicle Routing Problem (*VRP*) and the Job Shop Scheduling Problem (*JSSP*) — which are extensively researched, but do not fully address the requirements of the use case described in Section 1.4. However, key ideas and concepts from these fields offer valuable insights for developing *MRTA* solutions.

2.1. Vehicle Routing Problem (VRP)

First introduced in 1959 [28], the classic *VRP* aims to assign vehicle routes to visit each given location exactly once and return to the depot. The *VRP* is closely related to the Traveling Salesman Problem (*TSP*), but it involves optimizing routes for multiple vehicles instead of just one [29], making it more similar and applicable to *MRTA*. The classic *VRP* involves a fixed number of homogeneous vehicles, starting and ending their tours at the depot. Customers/Locations are assigned to vehicles, and the routes determine the order of visits while ensuring that the total demand on any route does not exceed vehicle capacity. Usually, the objective is to optimize routing costs (distance/time), and most commonly, the total routing cost or maximum routing cost of one vehicle is minimized [30].

There has been research into extending the *VRP* to heterogeneous fleets [31–36] which has shown promising results and could also be used to model robotic fleets with varying capability. One approach to achieve this is through extending the *VRP* with an approach similar to the colored multi-*TSP* [37], where vehicle assignments are restricted to match location/task requirements, effectively capturing heterogeneity. However, there is no straightforward way and little research into modeling the dynamic coalition formation of agents/vehicles within the *VRP*. Furthermore, *VRPs* are primarily concerned with minimizing routing costs, while in the *MRTA* use case described in Section 1.4, the task execution time is of high importance. This cannot be directly modeled using the classic *VRP* formulation.

Although *MRTA* problems for visiting a set of target locations is a variant of the *VRP* [38], *VRP* formulations are not directly applicable to the use case described in Section 1.4, because task execution time, as well as dynamic coalition formation, are usually not modeled. Nevertheless, some of the concepts to solve *VRP* could also be used in *MRTA*, like the idea of learning to select the heuristic to use for a specific problem instance as in [39] or improving solutions with Large Neighborhood Search [40].

2.2. Job Shop Scheduling Problem (JSSP)

The *JSSP* aims to schedule a set of jobs, each containing sequential operations with specific processing times, to a set of machines to minimize overall makespan [41]. It is one of the most widely studied combinatorial optimization problems, due to its broad applications in engineering and social domains [42]. In its traditional form, the *JSSP* assumes no transport times between machines, requires each job to utilize all machines, and does not account for coalition formation.

The flexible job-shop scheduling problem (*FJSSP*) variant allows operations to be processed by multiple machines within a subset of eligible machines [43], similar to how any robot with the required skills

can perform a task in *MRTA*. Variants extending the *FJSSP* formulation with transport times between operations like [44], are not generalizable to *MRTA*, since they assume static machines with jobs moving between them, while in *MRTA*, the robots themselves (analogous to machines in *FJSSP*) are mobile.

Furthermore, none of the *JSSP* formulations can handle dynamic coalition formation, making them insufficient for fully modeling the use case described in section 1.4. Nonetheless, insights from the *JSSP* literature serve as inspiration for *MRTA* solution methods, like speeding up the training process in RL by constraining the action space with predefined dispatching rules as in [45] or graph-based imitation learning from an expert system [46]. The survey by Smit et al. [43] identified a clear trend from 2022 to 2024 of using GNNs to address the *JSSP*, suggesting their potential applicability for the use case described in Section 1.4.

3

Discussion of Modeling Assumptions

This chapter presents the fundamental assumptions for modeling different *MRTA* aspects, covering heterogeneity, task generation/decompositions, coalition formation, task constraints, and uncertainty.

3.1. Heterogeneity

This section describes the main approaches to model heterogeneity in *MRTA*, as defined in Section 1.1. The literature mainly uses four approaches for this: *Binary*, *Mild*, *Capability with Execution Quality* (combining *Binary* and *Mild* for advanced modeling) and *Stochastic Capability Modeling*.

Binary Heterogeneity [19, 47–50]

Binary heterogeneity models robots' capabilities and task requirements as binary vectors enabling task assignment based on discrete skill presence.

Each robot r_i is characterized by a binary capability vector $\mathbf{c}_i^r \in \{0,1\}^K$, where K represents the total number of possible capabilities. Each element in $\mathbf{c}_i^r$ indicates whether the robot possesses a specific capability. For instance, if the capabilities are $[\text{Ground Navigating}, \text{Grasping}, \text{Flying}]$, a robot with $\mathbf{c}_i^r = [1,0,0]$ can navigate on the ground but cannot grasp objects or fly. Tasks are similarly defined by their requirements. Each task t_j has a capability requirement vector $\mathbf{c}_j^t \in \{0,1\}^K$, which is a one-hot vector indicating the specific capability needed to execute the task. $A_{i,j}$ is the scheduling matrix, where $A_{i,j} = 1$ if robot i is assigned to task j, and $A_{i,j} = 0$ otherwise. The combined capabilities of all robots assigned to task t_j must satisfy the capability requirements $\mathbf{c}_j^t$:

$$\left(\bigvee_i A_{i,j} \cdot \mathbf{c}_i^r \right)^\top \cdot \mathbf{c}_j^t = \|\mathbf{c}_j^t\|_1 \tag{3.1}$$

Binary heterogeneity is used in [47] to solve the bipartite matching based on the estimated reward as given in Equation (3.1). Similarly, [48] employs this concept to construct a binary mask, which filters rewards to retain only those corresponding to feasible assignments. In the Ant Colony Approach (*ACO*) in [19] an ant will only choose a task if it can contribute a skill. It will then wait until enough ants are assigned, so the binary capabilities of the coalition match the task requirements. Chakraa et al. [49] also use binary heterogeneity to model robots with differing sensing capabilities for their genetic algorithm solution, which only allows valid robot-task pairs in the chromosomes.

Aswale et al. [50] use binary heterogeneity in their *MILP* formulation like:

$$\forall s, \forall k \in \{0, m+1\}, \sum_{i=0}^{n} \sum_{j=0}^{m} x_{ijk} q_{is} \geq r_{ks}. \tag{3.2}$$

Where s are skills, k are tasks, x_{ijk} means robot i is attending task k after finishing task j, q_{is} is the capability of robot i to do skill s and r_{ks} is task k requires skills s.

Mild Heterogeneity [51–53]

Mild heterogeneity refers to a straightforward approach to modeling heterogeneous robots, where every robot i can execute any task j, which has a defined workload w_j. However, the execution time for each task varies among robots due to differing execution efficiencies c_i, leading to task durations dur_{ij}. This formulation captures the variation in performance across robots, but does not account for certain robots not being able to execute certain tasks.

Both [51] and [52] adopt this mildly heterogeneous model, which can be expressed as:

$$\text{dur}_{ij} = \frac{w_j}{c_i}, \tag{3.3}$$

This formulation inherently implies that the task duration dur_{ij} for a given task j does not have to be the same for each robot in general (Equation 3.4).

$$\text{dur}_{ij} \neq \text{dur}_i \tag{3.4}$$

In [53], human agent's task execution times are modeled to be varying over time, approximating their learning curve. As agents complete tasks across rounds, the estimator models task execution speed as a Gaussian distribution with noise decreasing exponentially with repeated task completion.

Heterogeneity in Capability with Execution Quality [13–15, 21, 54, 55]

Heterogeneity in Capability with Execution Quality combines binary capability presence with continuous efficiency/resource measures for advanced heterogeneity modeling.

In [55] each of the m tasks has specific requirements for both binary capability presence c_j^{tc} (for k capacities) and capability quality modeled as a resource requirement c_j^{tr}, robots have individual capacities c_i^{rc} and resources c_i^{rr}. Coalitions of robots are formed to meet both the capability presence and resource requirements of tasks, as in Equation 3.5 and 3.6.

$$\sum_{r_i \in A_j} c_{di}^{rc} \geq c_{dj}^{tc} \quad \forall d \in \{1, \ldots, k\}, \quad \forall j \in \{1, \ldots, m\} \tag{3.5}$$

$$\sum_{r_i \in A_j} c_{di}^{rr} \geq c_{dj}^{tr} \quad \forall d \in \{1, \ldots, k\}, \quad \forall j \in \{1, \ldots, m\} \tag{3.6}$$

In the bidding process of Irfan et al. [54], each robot r_i has a capability matrix that describes its quality q_{ik} and cost c_{ik} for each task k. On the other hand, tasks t_j have specific requirements to be fulfilled for execution. Each task t_j has a requirement matrix that specifies the required capabilities h_{jk} and the quality requirements w_{jk} for those capabilities. In the auction process to assign tasks, robots calculate their bid based on their ability to meet the task's workload and quality demands, with high costs having a negative effect on the bid. Bischoff et al. [15] enumerate all possible robot coalitions and assign a task duration to each coalition-task pair. The duration depends on the robots' completion rate and is set to ∞ for coalitions incapable of performing the task. This ensures that such coalitions are excluded from being assigned the task, as their heuristic prioritizes the coalition that minimizes the increase in the cost function. Similarly, the CoLoSSI auction-based framework [14] characterizes each robot k by its task performance function $\phi_k(i)$ for task i which defines the agent's ability and efficiency to complete a task i. This performance function influences the bid that each robot gives for a task. Muhuri et al. [21] incorporate the presence and quality of capabilities in coalitions to ensure they meet task requirements. In their genetic algorithm, coalitions that fail to satisfy these requirements are assigned a fitness value of zero in the evaluation function. Tercio [13], models agent capabilities defining the subtasks each agent is qualified to perform, along with the agent's minimum, maximum, and expected time required to execute each subtask.

Stochastic Capability Modeling [20]

Stochastic capability modeling accounts for uncertainty in robot capabilities and task requirements, enabling robust *MRTA* [20]. In this approach, the capabilities of each agent species $k \in V$ are represented as a non-negative capability vector $\mathbf{c}_k = [c_{k1}, \ldots, c_{kna}]^\top$, where c_{ka} is a random variable with any known distribution. Similarly, task requirements γ_{ia}, representing the required capability a for task i, are also modeled as random variables with known distributions.

A task requirement function $\rho_i(\cdot)$ evaluates whether the team's combined capabilities meet the requirements for task i. It is a binary function using logical operators to combine cumulative and non-cumulative capabilities. For example, cumulative capabilities (e.g., payload) are computed as the sum of contributions from all agents, while non-cumulative capabilities (e.g., speed) depend on the minimum capability among agents. The capabilities α_a of an agent team a are defined as:

$$\alpha_a = \begin{cases} \sum_{k \in V} c_{ka} \cdot y_{ki}, & \text{if } a \text{ is cumulative,} \\ \min_{k \in V} c_{ka} \cdot r_{ki}, & \text{if } a \text{ is non-cumulative,} \end{cases}$$

where y_{ki} is the number of agents of species k assigned to task i, and r_{ki} is a binary variable indicating if any agent of species k is assigned to task i.

For cumulative capabilities, the task requirement constraints are formulated as:

$$1 = \rho_i \left(\sum_{k \in V} E(c_{k1}) y_{ki}, \sum_{k \in V} E(c_{k2}) y_{ki}, \ldots \right), \quad \forall i \in M,$$

and for non-cumulative capabilities:

$$1 = \rho_i \left(\min_{k \in V} E(c_{k1}) r_{ki}, \min_{k \in V} E(c_{k2}) r_{ki}, \ldots \right), \quad \forall i \in M,$$

where $E(\cdot)$ represents the expectation operator, M is the set of all tasks. This deterministic constraint is added, to drive the task requirement function $\rho_i(\cdot)$ to be equal to 1.

3.2. Dynamic Coalition Formation

As detailed in Section 3.1, to be able to perform a task, the coalition members - collectively - must satisfy the task requirements. This can be modeled through *Atomic Tasks* or *Non-atomic Tasks*.

Atomic Tasks [9, 20, 21, 24, 47, 48, 50, 54, 55]

Atomic tasks require all robots that belong to a coalition to be present at the task location simultaneously for the entire task duration. Since robots may arrive at different times, task execution waits until all other robots arrive at the task location [19, 50]. Coalitions for atomic tasks can either be implicitly or explicitly defined: Babincsak et al. use both approaches in their *ACO* method [19]: In their TACO algorithm variant, coalitions are implied by the individual robot's schedules while the SAS variant assigns coalitions explicitly in the solution construction. Gao et al. [48] assign tasks explicitly to coalitions. For tasks that cannot be completed by a single robot, all possible coalitions capable of performing them are enumerated. In [50] coalitions in the *MILP* solution are defined implicitly through individual schedules, that are adjusted to wait for the last coalition member to arrive before starting execution. In [54] a robot is selected as the leader for a task and evaluates the required work to complete the task. It then forms a coalition by initiating auctions to nearby robots, selecting coalition members based on their capacities to collectively meet task demands. In [15] the constructive heuristic explicitly assigns tasks to the robot coalition, that increments the objective function the least.

Non-atomic Tasks [14, 56]

Ansari et al. claim to be the first to introduce non-atomic tasks in 2024 [14], although [56] propose a very similar approach in 2022. Non-atomic tasks allow robots to incrementally perform tasks with multiple

individual contributions that do not have to take place simultaneously. Unlike atomic tasks, where all coalition members must be present before execution begins, non-atomic tasks can start immediately and proceed as participating robots arrive. In [14] each robot is described by linear and additive performance functions, indicating how much a given task's workload decreases per time step spent on it. Correspondingly, each task has a completion map that specifies the fraction of remaining workload at any given time. [56] also allows for incremental execution. Here, task execution time is dependent on coalition size. Bigger coalitions are faster at execution, but too many allocated robots can also slow down execution due to congestion.

3.3. Task Generation and Decomposition

The generation and decomposition of tasks into elemental sub-tasks - including locations, requirements, and durations - has to precede the task allocation in MRS [2]. Most *MRTA* methods in the literature rely on tasks being presented in a well-defined form suitable for direct input into their scheduling algorithms. Some *MRTA* papers explicitly acknowledge that their task sets are the outcome of a prior decomposition process [57]. Most other papers simply state the assumption that the task set is known and provided without detailing its origin, e.g. [19, 53, 55]. An exception is [20], which models task requirements as probability distributions. This approach is more realistic for scenarios where a perception module generates tasks, reflecting the uncertainties inherent in real-world environments.

In practice, tasks must either be manually defined by a human expert or generated autonomously through an advanced perception and reasoning system. Such a system processes sensor inputs and high-level instructions to automatically create tasks, as demonstrated by Liu et al. in [58]. They employ a scene encoder to generate semantic maps from RGB-D images, enabling an attention-based planner to break down high-level language instructions into key actions and subtasks using the semantic map. There are also knowledge reasoning frameworks like [59], that aim to enable task planning based on perception of the environment.

Recently, there have been developments of using *LLM*s in combination with perception to decompose and generate tasks as in [60] which uses an *LLM* as a central planner that, given a high-level instruction and each robot's observed environment, decomposes the instruction into subtasks. Perception inputs (like scene graphs) and each robot's action set are converted into text prompts fed to the *LLM*, which proposes a task plan and assigns subtasks.

3.4. Task Constraints

This section explores how task constraints are modeled, analyzing *Precedence*, *Deadline*, *Waiting*, and *Battery Level* constraints, as well as *Priority Levels* between tasks.

Precedence Constraints [13, 15, 20, 47, 48, 50, 55, 56]

Precedence constraints ensure that certain tasks are completed before others can begin, reflecting dependencies and logical (partial) ordering of tasks [23].

According to the authors of [56] the three most common ways of modeling precedence constraints are:

1. Using mathematical models that define the dependencies between tasks, as in [13, 20, 50]. For a task t_j, which is the successor of task t_i, one can enforce, that t_j can only start after t_i finished and the corresponding robot traveled between the two tasks:

$$t_{start}^{(j)} \geq t_{start}^{(i)} + t_{execution}^{(i)} + t_{travel}^{(ij)} \tag{3.7}$$

2. Categorizing tasks into executable and non-executable groups based on precedence, offering a simpler method [15, 55], that might lead to suboptimal performance. Instead of waiting until a task enters the executable group, it may be better to move robots toward it in advance, when the predecessor will finish soon.
3. Directed Acyclic Graphs (*DAG*) are used to model precedence relationships between tasks. Recent advancements incorporate graph neural networks (*GNN*) to leverage the structural information within *DAG*, improving the handling of complex precedence constraints [47, 48, 56]

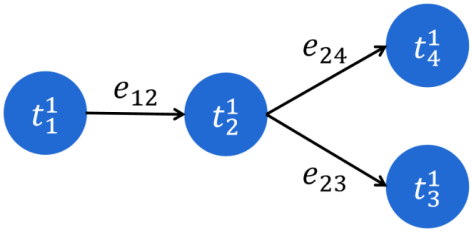

Figure 3.1: Simple *DAG* showing precedence constraints between tasks from [56].

Deadline Constraints [13, 51, 52]

Deadline constraints can be defined in absolute terms [51, 52], requiring that task t_i finishes before the deadline $T_{deadline}$ as in 3.8.

$$f_i \leq T_{deadline} \tag{3.8}$$

Deadline constraints can also be modeled relative to another task [13] where finish time f_j of task t_j, must be within the specified deadline $T_{deadline}$ relative to the start time s_i of task t_i, as in equation 3.9.

$$f_j \leq s_i + T_{deadline} \tag{3.9}$$

Waiting Constraints [13, 51, 53]

Waiting constraints ensure that a certain amount of time T_{wait} must elapse between the completion time of task t_i which can be expressed as f_i and the start of task t_j given as s_j, as in Equation 3.10.

$$s_j \geq f_i + T_{\text{wait}} \tag{3.10}$$

Priority Levels [61]

To prioritize urgent tasks, the authors of [61] assign higher reward values to priority tasks, directly influencing the *ACO* algorithm's objective to ensure agents are preferentially allocated to these tasks.

Battery Level Constraints [20, 49]

Battery level constraints ensure that the energy usage of robots remains within their capacity. Following the formulation in [20], the maximum cumulative energy g_{ki} spent by robot k at node i is modeled using the following constraints:

$$g_{ki} - g_{kj} + b_{kij} \leq B_{\text{large}}(1 - r_{kij}), \quad \forall i, j \in N, \forall k \in V, \tag{3.11}$$

$$g_{ki} \leq B_k, \quad \forall i \in N, \forall k \in V, \tag{3.12}$$

where b_{kij} is the energy cost for robot k to travel edge (i, j), r_{kij} indicates whether edge (i, j) is traveled, B_k is the energy capacity for species k, and B_{large} is a large constant for the *MILP*.

Equation (3.11) ensures that the cumulative energy g_{ki} increases by b_{kij} when traveling an edge, while equation (3.12) ensures the energy usage does not exceed the agent's capacity B_k. These constraints focus on the most costly path in the flow network and balance scalability by modeling energy constraints at the species level rather than individual agents, trading off precision for efficiency.

3.5. Uncertainty

Two primary sources of uncertainty in *MRTA* problems arise from the stochastic nature of *Execution Times* and *Travel Times*. In addition to these, uncertainty can also be modeled by representing robot capabilities and task requirements as random variables, as previously discussed in Section 3.1.

Stochastic Execution Times [13, 53, 56]

Deng et al. emphasize the difficulty of mathematically modeling the exact characteristics - and therefore execution times - of tasks [56]. They model task execution times as coalition-dependent Gaussian distribution. Their proposed *RL* method relies on observing these stochastic execution times during training, allowing it to implicitly learn to handle uncertainties. Consequently, the probability distributions of execution times should remain consistent between the training and testing phases for effective performance.

[53] deals with human-robot teams and models human task execution times as stochastic. Human task execution times are sampled from distributions that evolve according to a learning curve model, where performance improves as the human repeats a task but remains inherently uncertain. This is achieved by having the environment simulate human execution times using a probabilistic learning-curve estimator.

The Tercio algorithm [13] does not explicitly incorporate stochasticity in its optimization. Instead, it handles uncertainty through flexible time windows and rapid re-optimization. Each task duration is represented with agent-specific minimum and maximum execution times, returning task execution intervals that allow for variation in actual completion times. The lower bound ensures that the subtask duration is never less than $lb_{a\tau_{i,j}}$, while the upper bound prevents it from exceeding $ub_{a\tau_{i,j}}$. Here, $f_{i,j}$ and $s_{i,j}$ denote the finish and start times of subtask $\tau_{i,j}$, and $A_{a\tau_{i,j}}$ is a binary decision variable indicating whether agent a is assigned to the subtask and M is a sufficiently large constant.

$$f_{i,j} - s_{i,j} \geq lb_{a\tau_{i,j}} - M(1 - A_{a\tau_{i,j}}) \tag{3.13}$$

$$f_{i,j} - s_{i,j} \leq ub_{a\tau_{i,j}} + M(1 - A_{a\tau_{i,j}}) \tag{3.14}$$

Stochastic Travel Times [50]

In [50], uncertainty is modeled through stochastic travel times for robots between tasks. It is modeled by adding a safety margin t^s to the ideal travel time, to ensure arriving at a task with a given probability ϵ. To derive that safety margin, the expected delay is captured as Gaussian noise with mean μ and standard deviation σ. Specifically, if t denotes the ideal travel time between two tasks, then:

$$t^* = t + \mathcal{G}(\mu, \sigma). \tag{3.15}$$

For each robot in a coalition, we can express the need to arrive at the task as:

$$P(t^* < \bar{t}) \geq \epsilon, \tag{3.16}$$

where $\bar{t}$ is the hypothetical starting time. Using the inverse of the cumulative distribution function $\Phi(\cdot)$ of $\mathcal{G}(0, 1)$, the safety margin t^s can be expressed as:

$$t^s = \mu + \sigma^2 \Phi^{\text{inv}}(\epsilon), \tag{3.17}$$

4

Existing Solution Methods from the Literature

In the following chapter, all reviewed methods must address either Scheduling of heterogeneous robots (Section 4.1), dynamic coalition formation (Section 4.2), or both (Section 4.3). Papers that cover neither are excluded in this chapter but considered in the discussion (Section 3).

4.1. Heterogeneous Robots

This section analyzes papers that solve heterogeneous *MRTA* without dynamic coalition formation.

Optimization-based

Gombolay et al. introduced the centralized Tercio algorithm in [13], capable of handling complex temporospatial constraints (e.g., precedence, deadlines, wait) and scheduling heterogeneous robots. However, Tercio's main limitations are its inability to form coalitions and its assumption that travel times between tasks are negligible. The scheduling algorithm is split into two stages:

1. *MILP Task Allocator* with exponential runtime, assigns tasks to agents, optimizing makespan and balancing maximum workload per robot. The allocations serve as input to the Task Sequencer.
2. *Task Sequencer* with polynomial runtime, generates valid schedules for all robots, by focusing only on satisfying constraints rather than optimizing an objective function. This degrades solution performance but significantly improves runtime. It uses an online schedulability test to verify if a subtask can be scheduled at a given timestep while meeting the task constraints.

Their solution, inspired by the real Boeing 777 assembly process, is tested on a synthetic dataset and compared against the exact solution, an auction-based method [62], and the genetic algorithm OCGA [63]. (Note: These algorithms are not detailed here as the survey already covers more recent, similar approaches). The algorithm achieves computation times of under 1 second for problems with 5 robots and 49 tasks (CPU: 8 cores @ 3.20 GHz, 16 GB RAM), making it 10x faster than the auction-based method and approximately three orders of magnitude faster than the exact and *GA* methods, with an optimality gap of less than 10% relative to the exact solution. This performance surpasses previous state-of-the-art methods and still serves as a comparison in recent papers [51, 64].

Auction/Market-based

The authors of [14] introduce a distributed auction-based method for scheduling heterogeneous robots called CoLoSSI, treating tasks as non-atomic, implying that any task can be completed incrementally over separate periods with contributions of different robots. This extends their framework in [65] to restricted-communication environments. The method uses sequential auctions with one auctioneer broadcasting tasks and agents bidding on tasks based on factors like task duration, duration of current schedule, and travel distance. The auctioneer assigns tasks to agents with the lowest bids and updates all agents on mission progress, resulting in schedules like in Figure 4.1. Subsequently, agent

schedules are refined by sharing overlapping tasks ("Iterative Collaborative Refinement", Figure 4.2) and balancing workloads ("Makespan Balancing and Refinement", Figure 4.3) to minimize makespan. Resulting schedules can have two robots working together on the same task simultaneously, however, this does not satisfy the definition of dynamic coalition formation given in chapter 1.1, as coalitions are not formed on purpose. The authors implement an example search and rescue mission with a heterogeneous team and report an optimality gap of 10% for a team of 6 robots compared to the exact solution. Furthermore, the method achieves 50% improvements in makespan compared to other state-of-the-art Sequential Single Item Auctioning algorithms.

In the figures below, the grid on the left illustrates the allocation of tasks to agents, while the graph on the right depicts the time units each agent will dedicate to their respective tasks.

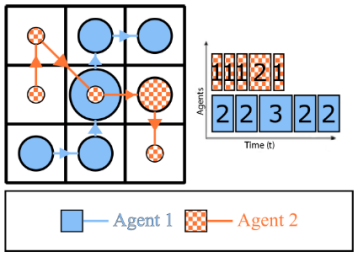

Figure 4.1: Initial task allocation after auction from [14].

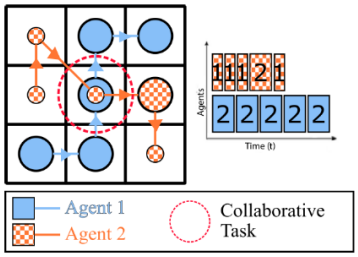

Figure 4.2: Sharing overlapping tasks - "Iterative Collaborative Refinement Step" from [14].

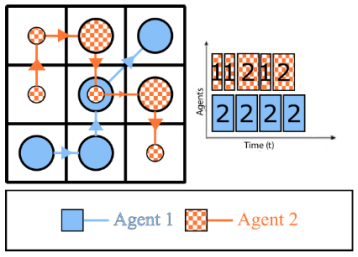

Figure 4.3: Optimized schedule - "Makespan Balancing and Refinement Step" from [14].

Graph Learning

Wang et al. propose ScheduleNet in [51], a model leveraging a heterogeneous *GNN* (HetGAT) to learn scheduling policies for heterogeneous *MRTA* problems without coalition formation. This paper extends their earlier work in [64] to mildly heterogeneous robots - differing in task completion proficiency and therefore task execution time. The problem is represented by a Simple Temporal Network (*STN*) which encodes robots, tasks, and current partial schedules as nodes and edges encode relationships such as *locatedIn* and *assignedTo*, along with temporal constraints like wait, deadlines, and proximity constraints to prevent collisions during task execution. The HetGAT approximates the Q-score of appending tasks to a robot's schedule using per-edge-type message passing (through different weight matrices) and per-node-type feature reduction (different attention coefficients).

They use the following *MDP* formulation for the problem:

- *State: STN* of the problem and all robots' partially constructed schedules at the given timestep
- *Action:* Corresponds to appending an unscheduled task at the end of a partial robot schedule
- *Reward:* Change in makespan after appending the task

Although this *MDP* formulation suggests the use of *RL* for training, as is done in [52], ScheduleNet uses Imitation Learning to accelerate training, as their formulation results in most schedules being infeasible, causing *RL* to waste significant time exploring them. They test on a synthetic dataset based on [66] and compare against the heuristic earliest deadline first (*EDF*, which assigns the earliest deadline task to the earliest available robot), Tercio [13], and the exact solution using the GUROBI[1] solver. The percentage of problems solved within a given optimality ratio w.r.t. the exact solution (r) is used as a performance criterion. ScheduleNet outperformed *EDF* and Tercio on small to medium problems (16-50 tasks, 2-5 robots) for $r \geq 1.2$, except for small problems with ten-robot teams. The improvement was most notable in medium-sized problems, where ScheduleNet found feasible solutions with $r \geq 1.1$ for over 50% of cases, compared to less than 15% for Tercio and *EDF*. ScheduleNet's computation time is significantly faster than the exact solution, but slower than Tercio and *EDF*, taking about 10s using an NVIDIA Quadro RTX 8000 GPU to schedule even a small problem (20 tasks, 5 robots), since the full HetGAT has to be inferenced for each decision step.

[1] https://www.gurobi.com/solutions/gurobi-optimizer/

In contrast to the centralized Imitation Learning approach of [51], Paul et al. introduce CapAM in [52] which uses a decentralized policy trained with *RL* to solve the *MDP* formulation of mildly heterogeneous *MRTA*. CapAM focuses on maximizing the number of tasks completed within their deadlines, considering only deadline constraints. To solve the *MDP*, it uses a task encoder based on Graph Capsule Convolutional Networks (*GCAPCN*) [67] to fuse local task properties (coordinates, deadlines) with global structural problem information, as shown in Figure 4.5. This representation serves as key-value pairs for an attention-based decoder, where the query comprises contextual information about the robot making the decision and its peers, with the decoder's outputs determining the robot's next action. For each robot, the *MDP* is formulated as:

- *State:* The robot's location and work capacity, planned tasks of the peers, and environment information like elapsed mission time and task features extracted using the *GCAPCN*
- *Action:* Corresponds to selecting the next task to execute in a decentralized way
- *Reward:* Negative reward for each task that was not completed within its deadline

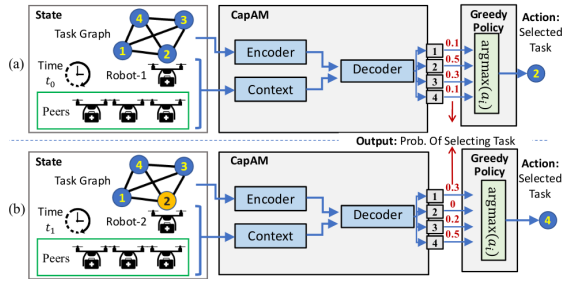

Figure 4.4: Deployment of an *MRTA* policy based on the CapAM architecture from [52].

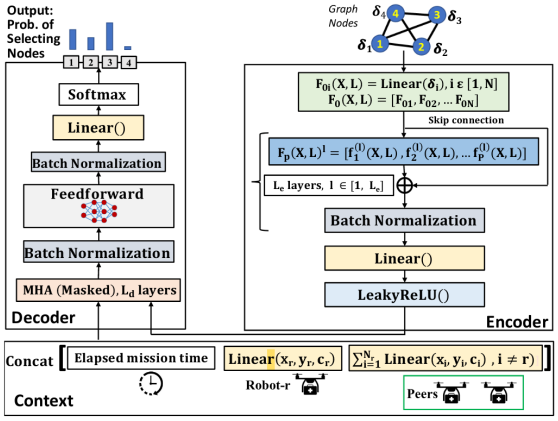

Figure 4.5: CapAM architecture according to [52] with Encoder, Decoder and Context.

CapAM is trained using REINFORCE [68] with a policy and baseline network similar to [9], explained here and tested on the dataset of [69] with 100 tasks per problem with deadlines, varying team size (2-7), workloads, and robot execution speeds. They compare against Iterated Local Search (ILS) [70], an enhanced version of that (EILS) [69], a bipartite graph matching technique called BiG-MRTA [71] and most interestingly the AM-RL method inspired by [72], which uses a similar architecture but with a purely attention-based the encoder. CapAM can complete more tasks within their deadlines compared to the baseline approaches, especially highlighting the superior performance of the GCAPCN encoder. Furthermore, CapAM can solve problems with 200 robots (twice as much as during training) better than the baseline methods with increasing performance gaps compared to AM-RL. It also achieved faster runtimes (<0.1s for 100-task problems), but the authors do not report on used hardware.

Another *RL*-based method to schedule mildly heterogeneous robot formation is HybridNet introduced in [53]. HybridNet comprises two main components: a heterogeneous graph-based encoder and a recurrent *LSTM* schedule propagator. The encoder, which extracts embeddings from the *STN* representation of the problem, is computationally intensive and is executed only once per scheduling round. The schedule propagator then iteratively schedules tasks by selecting one task-agent pair at a time. After each scheduling decision, the *LSTM* updates the agent and state embeddings with information from the decision, ensuring the system adapts dynamically until all tasks are scheduled, an overview can be seen in 4.6. The heterogeneity comes from scheduling human-robot teams, with humans having a time-varying performance (task completion rate), estimated using a Learning Curve Estimator, which predicts human agent performance using past task duration data. It employs a black-box model inspired by Liu et al. [73] to simulate a stochastic human learning process. As agents complete tasks across rounds, the estimator models performance as a Gaussian distribution with noise decreasing exponentially with repeated task completions. HybridNet iteratively schedules human-robot teams,

adapting to human task performance changes. Each round it assigns tasks to agents sequentially, checking schedule feasibility under problem constraints (wait, deadline). If any constraints cannot be satisfied, the unscheduled tasks are carried over to the next round for reassignment. The Multi-Round Scheduling is modeled as a *POMDP*:

- *State:* State of all agents (human, robot)
- *Action:* At reach round, actions are a complete set of task allocations, executed sequentially
- *Transition:* Executing the given schedule and advancing to the next scheduling round
- *Reward:* The total reward is computed as the sum of rewards from feasible task allocations and penalized estimates for infeasible allocations. This formulation prioritizes feasible schedules while enabling learning from infeasible assignments
- *Observation:* Estimated performance (by Learning Curve Estimator as Observation Function) of task-agent pairs and observable constraints.

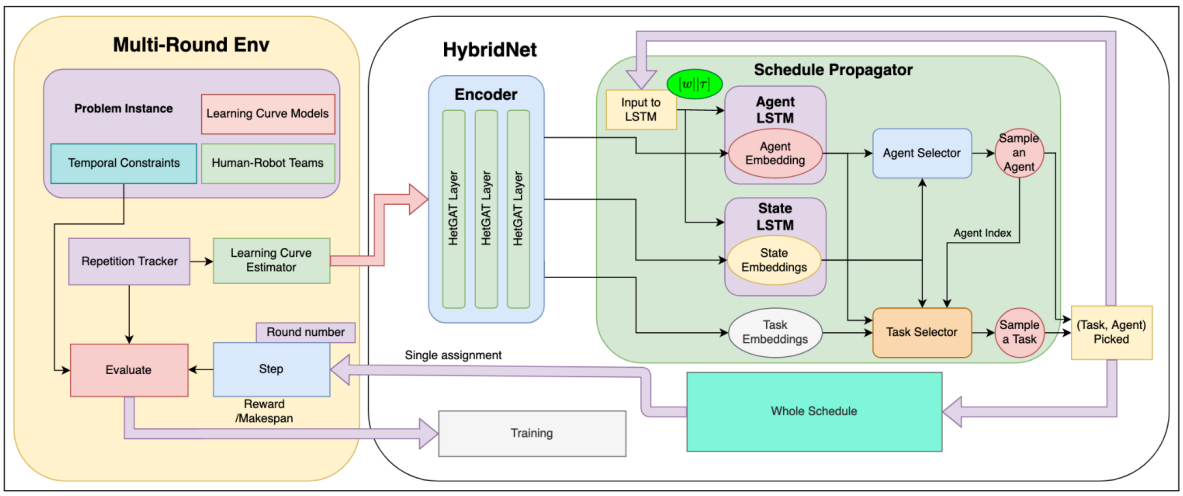

Figure 4.6: Overview of the HybridNet approach from [53] Left: The Multi-Round Scheduling Environment simulates a human-robot scheduling scenario, adapting to changes in human task performance. Right: HybridNet features a heterogeneous graph-based encoder for high-level embedding extraction and a recurrent schedule propagator for rapid schedule generation.

HybridNet is trained using Policy Gradient methods, optimizing based on the advantage term compared to a baseline, similar to [56]. To test, the authors generate problems across different scales, each with 2 humans and 2 robots in the team, but differing in task number (small: 9-11, medium: 18-22, large 36-44), with 25% of tasks including wait or deadline constraints. They compare against the earliest deadline first heuristic (*EDF*), and a *GA* that improves the *EDF* solutions [74]. Additionally, they introduce HetGAT, a model that omits the *LSTM* schedule propagator and directly schedules task-agent pairs using the encoder's output. Over all three tested problem scales, HybridNet outperforms both *EDF* and the *GA* in makespan and percentage of feasible schedules generated. While the *LSTM*-based HybridNet and HetGAT achieve comparable performance, the *LSTM* version is over an order of magnitude faster during training and at least twice as fast during testing, however, it still requires about 11s to schedule a small problem instance on an NVIDIA Quadro RTX 8000 GPU.

Population-based

Ma and Gong present a hybrid approach, GHNN-ACO, which combines a heterogeneous *GNN* (GHNN) and *ACO* in [61]. In this approach, the *GNN* captures task-agent relations and informs the *ACO* search process, by enhancing the traditional pheromone update with learned task-agent representations to assign tasks to robots more efficiently. More specifically, the GHNN outputs guide the *ACO* algorithm and speed up the convergence. The GHNN's input is a graph with agents' states and task features as nodes and their communication/collaboration relationship as edges. The method was tested in an emergency rescue scenario involving 10 tasks assigned to drones, ships, and ground vehicles. The

authors report high accuracy in assigning the correct agent to each task but note significant computational costs. However, they do not include standard *MRTA* metrics, such as optimality gaps, makespan, or runtime analysis, and their results lack a clear explanation. Although GHNN-ACO cannot handle dynamic coalitions, it would be interesting to extend Babincsak's dynamic coalition-capable *ACO* method (as in [19]) with a similar *GNN*-informed pheromone update rule.

Another population-based method for the heterogeneous *MRTA* was proposed in [49], using a *GA*. Each candidate assignment (gene) encodes three attributes: task position, associated task requirement, and assigned robot. Combining all genes to cover all assignments forms the chromosome, representing a potential candidate solution. They seed their solution population randomly and apply crossover (merging of two chosen parent chromosomes) with a high probability and mutation (minor random changes to a chromosome) with a lower probability. Furthermore, they include a portion of the fittest parents when forming the new population. This enables the *GA* to balance exploration and exploitation as the population iteratively evolves. The authors evaluate their approach on a sensing mission, where *UGV* and *UAV* equipped with heterogeneous sensors perform different measurements at specific locations. Notably, they allow each robot to take several measurements simultaneously (*MT*). A comprehensive study used an Intel CPU i5 @ 2.6GHz, comparing the proposed *GA* with Hybrid Filtered Beam Search (*HFBS*), and an optimal CPLEX solver. For medium-sized problems (13-15 tasks, 3 robots, 3-4 skills) the proposed *GA* produces optimal solutions within 0.15 seconds, much faster than the optimal solver and providing better solution quality than *HFBS*. However, the *GA*'s performance declined with larger problem sizes (30-33 tasks, 4 robots, 4-5 skills), generating comparable non-optimal solution quality to *HFBS* while taking significantly longer to run.

4.2. Dynamic Coalition Formation

This section reviews MRTA approaches with homogenous robots that form dynamic coalitions.

Graph Learning

In [56], Deng et al. propose a centralized scheduler that integrates a *GAN* with a *RL* policy network. This approach models both precedence constraints and uncertainty in task execution times. Coalition formation is addressed by grouping tasks with precedence constraints represented as a *DAG*, while task execution times follow a Gaussian distribution. They do not require all robots to be present simultaneously to complete a task, similar to [14]. The *GAN* has a hierarchical attention mechanism, generating task embeddings that are aggregated into task group embeddings and combined into a global embedding, see Figure 4.7. All three embeddings are fed into the policy network that selects the next task and the number of robots to schedule for that task using MLPs. The model is trained using the REINFORCE algorithm [68] with the following *MDP* formulation:

- *State*: Represents unfinished task groups (*DAG*), executable tasks, and available robots, with tasks described by workload and robot assignment features.
- *Action*: Defines task allocation via action pairs (task, parallel robots)
- *Reward*: Aims to minimize average task group completion time

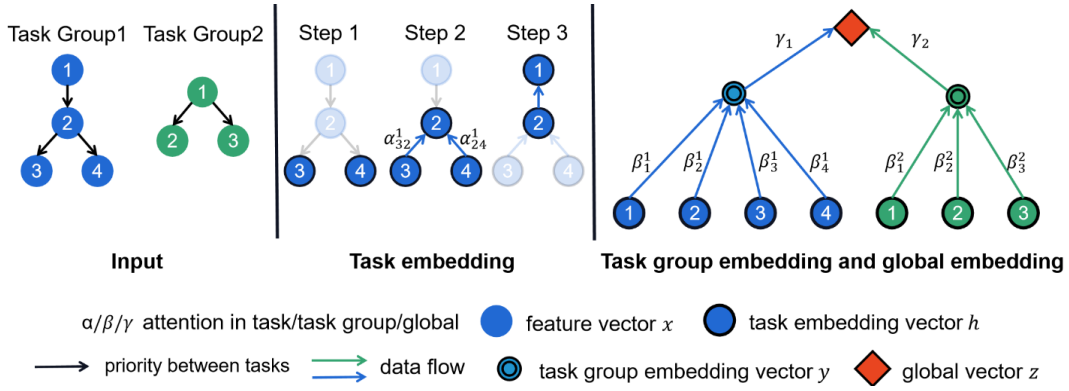

Figure 4.7: Hierarchical attention mechanism, generating task embeddings from [56].

To test their approach, Deng et al. compare against a random policy, the greedy heuristic from [15] and Min-Interfere [75]. The proposed method outperforms all three baselines across different problem scales (5-20 task groups including 2-15 tasks each, 5-20 robots). For runtime, the authors report 5.9ms per task assignment even for larger problems, using an Intel i9-9900k CPU and NVIDIA 3090 GPU. Notably, the method generalizes to problems of scales larger than seen during training (i.e. 80 task groups), while still outperforming the baselines.

Dai et al. also cast the homogeneous *MRTA* with dynamic coalitions as an *MDP*, but they use decentralized *RL* policies to solve it in [9]:

- *State:* Each robot observes an augmented graph representing task states relative to itself, the state of all other robots, and a binary mask indicating task completion. The graph includes task-specific attributes like coordinates, number of required robots, durations, and agent assignments, while agent states include relative positions, task progress, and assignments.
- *Action:* At each timestep, an agent's decentralized neural network generates a stochastic policy over tasks, filtered by the binary mask. Actions are sampled from the distribution during training and chosen greedily during inference.
- *Reward:* The objective is to minimize the makespan, with the reward equal to the negative makespan given at the end of each episode, with timeouts implemented to prevent deadlocks due to poor coordination in early training.

The policy network to select the next task is an attention-based encoder-decoder architecture shared among agents. The task, agent and cross encoders use multi-head attention and the decoder uses these outputs and decodes them into a probability distribution over tasks using single-head attention, as can be seen in Figure 4.8.

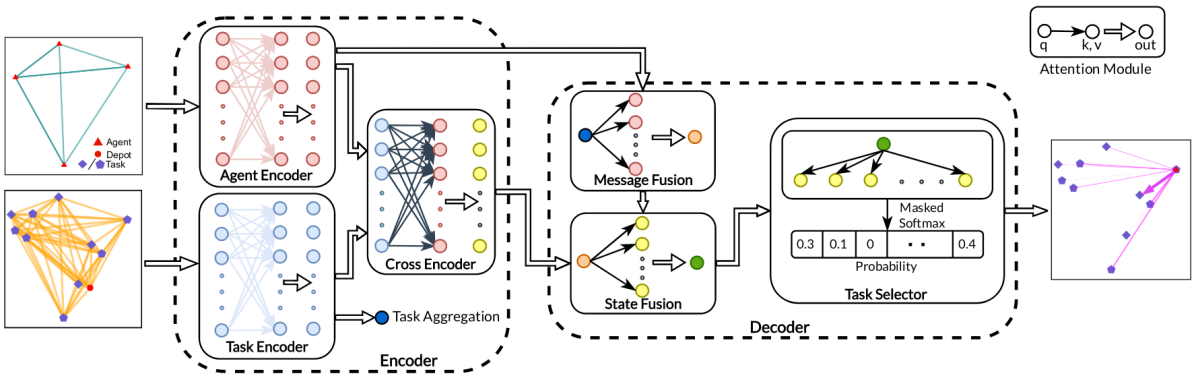

Figure 4.8: Overview of the approach used in [9]. Tasks and agent graphs are initially projected into embeddings and processed by a group of encoders to establish context. The decoder, fuses this context with the average task embedding to provide a global view of the system. A task selector then produces the probabilities, guiding agents in selecting tasks.

The training in [9] utilizes the REINFORCE algorithm [68] with two neural networks: a policy network and a baseline network. The final loss function is based on the reward difference between these networks. If the policy network consistently outperforms the baseline network, the baseline is updated with the policy network's parameters. Since agents in the same coalition will make the next decision simultaneously (since they finish the previous task at the same time), the authors propose a leader-follow mechanism during training, where the leader selects the next task, and robots from the previous coalition will follow that decision, if the next task requires it. This reduces decision complexity during training, leading to more efficient, low-variance policies, that will be executed in a decentralized manner during inference. The authors of [9] validate their method on a synthetic dataset against the exact CTAS-D solver [20] and meta-heuristic OR-Tools solver [2]. For small and medium-sized problems (10-20 agents, 20-50 tasks), the proposed method produces solutions with comparable performance to the exact solution (cutoff 300s), with two orders of magnitude faster computation time. Performance improves relative to the two baselines as the average number of robots required per task increases. On large-scale instances (40

[2] https://developers.google.com/optimization/

robots, 200 tasks), CTAS-D fails to produce a feasible solution within the time budget (3600s) and OR-Tools produces a slightly better solution (63.0s vs 64.3s makespan) but needs an order of magnitude more time. The authors do not specify the used hardware for the experiments.

Population-based

Arif proposes a genetic algorithm for the homogeneous *MRTA* with dynamic coalition formation, called RoSTAM (Robust and Self-adaptive Task Allocation for Multi-Robot Teams) in [24]. The method uses a two-part chromosome: The first encodes task execution order via task IDs and the second defines robot schedules from the task order. The task order is changed using the crossover operator and mutation (randomly switched between swap and inverse). Creep mutation is applied to the second part, which defines the assignment from task orders to robot schedules. Additionally, in each generation, random solutions are inserted into the population inspired by the artificial immune system from [76]. The fitness function evaluates the makespan and penalizes solutions with deadlocks - detected as cycles in the task allocation graph. The penalty factor is adaptive: it decreases if recent solutions are all feasible, increases if all are infeasible, and remains unchanged for a mix. This approach balances exploration and feasibility in the solution space. The method is applied to a firefighting mission where tasks require subsets of five distinct skills (e.g., water jet, gas detector). Certain tasks need multiple robots with a specific skill present simultaneously. The robot team is homogeneous, with all robots possessing all five skills but limited to contributing one skill per task. This makes skill heterogeneity irrelevant, as task allocation depends only on scheduling the required number of robots for each task, not the specific skills involved, since all robots have all skills. The provided solutions are compared against incremental and batch auction-based allocation methods and on varying problem sizes (10-50 tasks, 5-7 robots). RoSTAM outperforms the incremental method in all test scenarios and the batch method in 16/18 test cases. However, the author does not report on runtime, computational complexity or used hardware.

4.3. Heterogeneous Robots + Dynamic Coalition Formation

This section evaluates *MRTA* methods that model both heterogeneous robot capabilities as well as dynamic coalitions/sub-teams.

Exact formulation

Aswale et al. presented a complete *MILP* formulation for the heterogeneous *MRTA* with coalition formation in [50]. Their model incorporates stochastic travel times, which is valuable for real-world applications, as collision avoidance or unpredictable events along the path can cause deviations from assumed travel times [77]. A similar approach, but incorporating battery constraints and recharging into the formulation, has also been explored in [78], further enhancing the practicality of such models for real-world robotic systems. The exact formulation is optimal, but can only solve very small problems in an acceptable time. Even for small problem instances (8 tasks, 4 robots, 8 skills) the optimal solver required a median of about 300 seconds to determine an exact solution [50]. In extreme cases, it can take up to 5,000 seconds. This prolonged runtime was observed despite utilizing high-end hardware (AMD EPYC 7543 processors with 22 CPU cores and 156 GB of RAM). However, it is promising to serve as a baseline/benchmark - as in [19] - or generate ground truth solutions for learning algorithms.

Risk-based Stochastic Mixed Integer Program

In [20], Fu et al. present a method called CTAS (Capability-based robust Task Assignment and Scheduling) for heterogeneous *MRTA* with dynamic coalitions, addressing uncertainty in robot capabilities, modeled as vectors of random distributions. The task requirement function is a binary function that checks if a team's cumulative or non-cumulative capabilities meet a task's requirements, using logical operators. For a detailed description of the stochastic capability modeling, see Section 3.1. The approach emphasizes robust scheduling, creating plans with a higher likelihood of robot capabilities exceeding task requirements. However, this robustness increases redundancy and associated costs. The trade-off between redundancy and risk is optimized using *CVaR* [79] alongside makespan and energy consumption. In the first step, CTAS precomputes the best paths between all tasks using the A* algorithm [80] based on distance and other cost factors like viral exposure for their pandemic response test scenario.

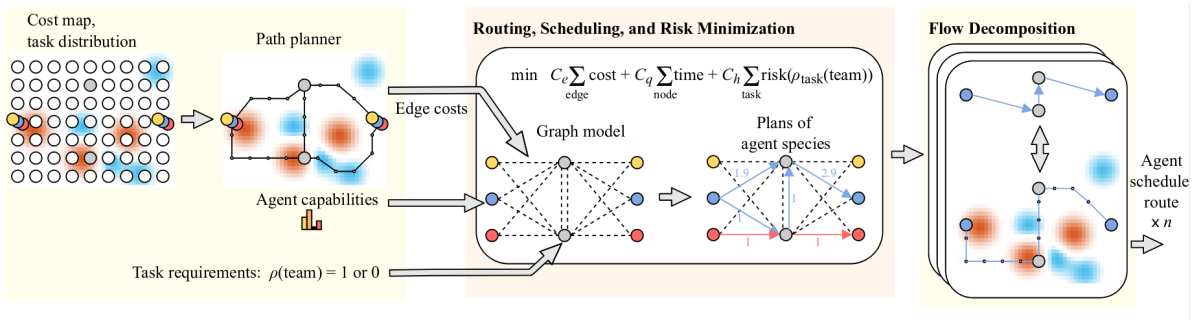

Figure 4.9: Overview of the approach used in [20], Inputs: path cost maps, agent capabilities, and task requirements. Outputs: agent routes, schedules, and team formations. The process includes network flow generation and flow decomposition.

The scheduling approach is visualized in Figure 4.9 and can be divided into two steps:

1. *Generate species flow with stochastic MIP:* Solving a stochastic *MIP* that outputs an agent flow between tasks for each robot species. The stochastic *MIP* is converted into a deterministic *MILP* by precomputing expectations, approximating stochastic terms using sample averages, and linearizing nonlinear functions with helper variables and piecewise linear constraints. It optimizes for makespan, energy consumption and robustness through the *CVaR* while respecting robot battery, precedence, and time window constraints
2. *Extract individual robot schedules* A Flow Decomposition algorithm is used to generate the individual schedules (see Figure 4.10), consisting of two steps:
   (a) *Rounding*: Convert fractional agent flows from the *MILP* solution into integer flows by rounding them up to satisfy task requirements while minimizing additional energy costs and maintaining flow constraints with an LP
   (b) *Splitting*: Decompose the rounded integer flows into individual agent routes such that the total flow matches the original rounded flows. An *ILP* minimizes the maximum individual agent energy cost while ensuring task and flow constraints are satisfied.

To evaluate CTAS, [20] uses a pandemic response scenario with 8 task types (e.g., delivery, disinfection), 9 capability types (e.g., flying, perceiving), and 7 agent species (e.g., quadcopters, treatment robots). The performance is compared to their earlier method, CTAS-O [81], which does not use flow decomposition, making the initial stochastic *MIP* harder to solve. Additionally, CTAS-D is tested, a variation of CTAS that excludes the risk term from the optimization. The analysis limits solvers to 120 seconds per test case to assess success rates using an Intel i7-7660U CPU with 2.50GHz. CTAS outperforms CTAS-O, by solving larger problems (up to 140 agents and 40 tasks) within the time limit. CTAS achieves a 35% increase in success probability compared to CTAS-O, with only a 20% increase in energy cost. It also demonstrates lower optimality gaps, solving problems with up to 24 tasks and 140 agents within a 3% optimality gap in 120 seconds. Removing the risk term, enables CTAS-D to solve even the largest test cases (40 tasks, 140 agents) with no optimality gap within the 120s time limit.

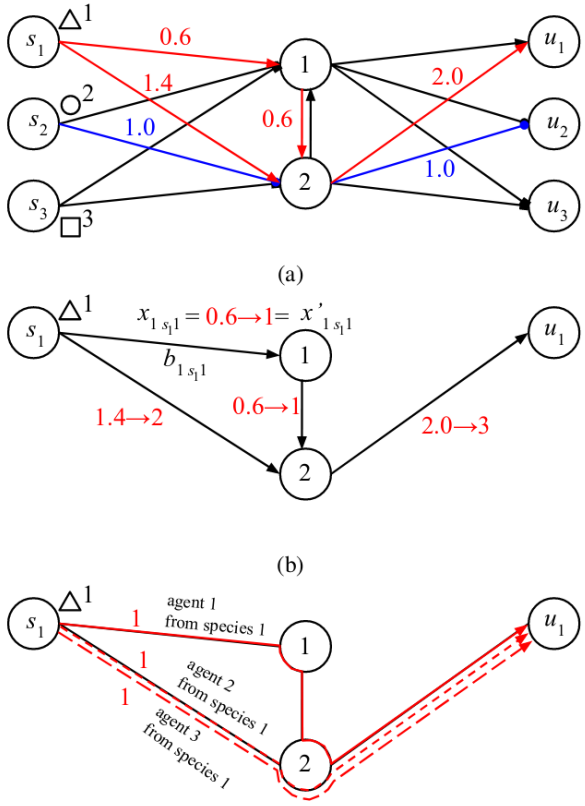

Figure 4.10: Visualization of the Flow Decomposition from [20]. (a) Agent flows with species distinctions (red and blue), (b) Flow of agent species 1 with rounding highlighted, and (c) Split of agent species 1 into individual paths for three agents.

Greedy Heuristics

To address the runtime limitations of their exact *MILP* formulation, the authors of [50] also introduce a fast though suboptimal, greedy heuristic to balance small coalition sizes (by reducing redundant skills in coalitions) with minimal travel times between tasks, to achieve low makespans. This leads to speedups of up to 5 orders of magnitude compared to the optimal solver while typically producing solutions with maximally double the cost. Note, that both the runtime improvement and the suboptimality of the greedy approach increase with problem complexity/size. Furthermore, the greedy solution can tackle large problems (1024 tasks, 32 robots, 64 skills) in less than 50 seconds, while the optimal solver fails to even produce a first solution within 3 hours for those scenarios.

In [15] Bischoff et al. propose an interesting approach of balancing the speed of greedy algorithms with improving their solution quality. They start with a construction heuristic, incorporating both static (e.g., task duration) and dynamic costs (e.g., transport energy, dependent on the schedule). The construction repeatedly assigns executable tasks to robot alliances by selecting the assignment that minimally increases the cost function, similar to the MinStepSum algorithm in [75]. After each assignment, it updates the set of executable tasks based on precedence constraints. This process continues until all tasks are assigned. The found solution is then refined using an improvement heuristic based on a novel neighborhood operator inspired by the relocate neighborhood technique from [82]. The heuristic attempts to improve cost by reallocating tasks between coalition schedules, making the algorithm anytime and suitable for real-time applications. The authors evaluate their approach using synthetic data with diverse robots and tasks across various problem sizes. Their improvement heuristic achieves an average cost reduction of 10%, increasing to up to 30% in more complex scenarios. Computation time (tested on Intel i58250U CPU at 1.6GHz) increases minimally for small instances (0.29s for 6 tasks, 4 robots) but can take significantly longer until full convergence for larger, more complex problems (36.58s for 15 tasks, 4 robots). The authors only report improvement relative to the initially constructed solution and not relative to the optimal solution.

Graph Learning + Bipartite Matching

Gao et al. formulate scheduling as a deep bipartite graph matching in [48], introducing a novel *GAN* to extract embeddings of the directed graph representing tasks and their relations, i.e. precedence constraints, as visualized in Figure 4.11. Whereas the authors of [47] also feed a robot graph into their *GAN* architecture, the robot embeddings in [48] are computed using an *MLP* with one layer. The final output of the network is the reward matrix of robot-task assignments, that is used to find the assignments that maximize the reward with bipartite graph matching. They model the robots as inherently heterogeneous, but coalition formation is only considered if a task fails to be completed by a single robot. In that case, their algorithm greedily searches for a suitable coalition of robots to complete the task. There might be edge cases, where a more optimal schedule could be obtained by considering coalition formation for all tasks, as in [47], not only for tasks that are not suited for a single robot (e.g. two robots are much faster at completing the task than one). The network is trained using Imitation Learning (*IL*) from optimal solutions on small-sized problems. *IL* is used to shift the online computational load of optimal solvers to an offline learning procedure [83]. The method shows better makespan and robot utilization compared to greedy and random baselines on synthetic data and Gazebo simulations. The approach scales well, maintaining high execution speeds of sub-second level even for large problems with hundreds of tasks and dozens of robots on a 16-core CPU (not specified which one) with 16GB of RAM. The authors do not mention using a GPU, which is unusual for learning-based methods. The runtime increases with the number of tasks/robots and the subteam size.

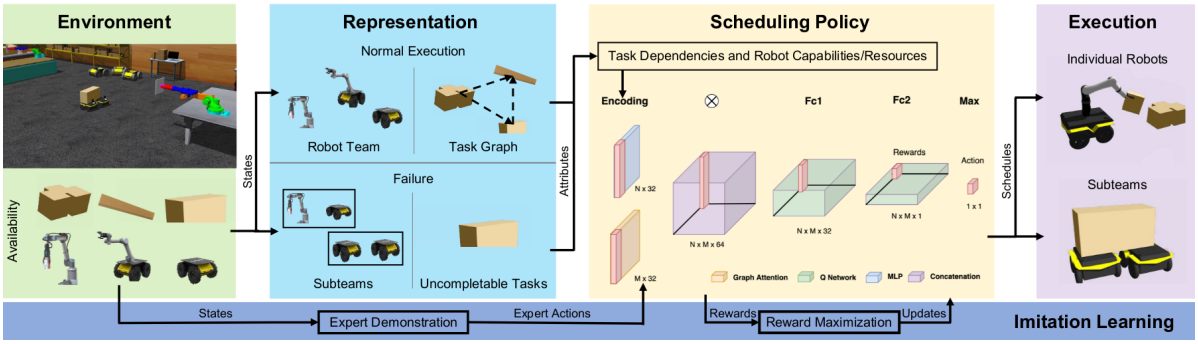

Figure 4.11: Overview of the approach used in [48]. Combines *GAN* on tasks graph with *MLP* on robots. If a task fails to be executed by a single robot, the algorithm searches for a sub-team (coalition) that can collectively be scheduled to the task.

Jose and Zhang [47] propose a similar architecture to [48] called LVWS (Learning for Voluntary Waiting and Sub-teaming), in which the concept of enabling robots to wait to form more optimal coalitions voluntarily is explored first. It also represents the problem as a bipartite graph matching, but with a *GATN* that uses the outputs of the *GAN* in a transformer to estimate the assignment reward matrix based on input robot and task graphs similar to [48], but extending it by also feeding a robot graph into a *GATN* module, as visualized in Figure 4.12. Subsequently, the optimal assignment is determined by finding the scheduling matrix, that maximizes the total reward. Importantly, they incorporate ”Wait” as an additional task option during scheduling, allowing robots to strategically delay actions to enhance team formation. It achieves dynamic coalition formation by relaxing the traditional one-to-one bipartite matching constraint, allowing multiple robots to be assigned to the same task. These robots must collectively meet the task’s requirements. They use *IL* from an exact solver to train the *GATN*. The LVWS approach was evaluated using synthetic datasets and Gazebo simulations using mobile bases and static manipulators. Compared with baselines (random, greedy, and the method in [48]), it achieves lower makespan suboptimality, especially excelling in large task sets. LVWS demonstrates scalability, handling scenarios with hundreds of tasks and 100 robots in less than a second, using a high-performance hardware setup (14-core Intel Core i7, 96 GB of RAM, NVIDIA RTX Titan GPU). Though execution speed decreases with the number of robots and tasks, LVWS remains viable for online scheduling.

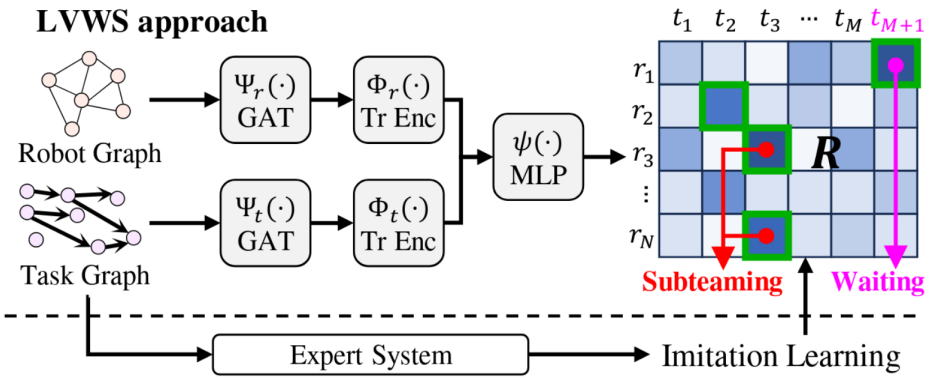

Figure 4.12: Overview of LVWS approach from [47] with the *GATN* embedding task and robot graphs, and transformer encoders and *MLP* estimating a reward matrix that is subsequently used for bipartite matching with sub-teaming and voluntary waiting.

Auction-based and Hedonic Games

Irfan et al. [54] introduce an auction-based method for problems with online incoming tasks with dynamic workload. When a new task is announced, each heterogeneous robot bids on the task based on its ability to contribute required skills, and the central auctioneer evaluates bids by weighing them against the cost for the robot to reach the task. The selected robot - the leader for that task - will start a decentralized auction (”recruiting”) to form a coalition with nearby robots until all required skills are covered. During execution, a task’s workload can change or coalition capabilities can deteriorate. In that case, task leaders can negotiate to swap robots between coalitions to re-adjust to the new situation. The discussion and results sections lacks key aspects such as performance/suboptimality, feasibility of generated schedules, and comprehensive runtime analyses. Since the algorithm is distributed, simple, and designed for online execution, it can be assumed that it can run in real-time. The authors present only a small experiment, where a coalition must negotiate replacing a team member with a more capable robot as the task workload suddenly increases during execution.

Dutta et al. are the first ones to use a distributed hedonic game formulation to address the heterogeneous *MRTA* with coalition formation in [84], in which robots aim to execute tasks based on their self-benefit. The paper by Zhang et al. [11] extends the idea by integrating multiple preferences (travel cost, expected number of robots for a task, task type) and addressing incomplete communication networks. Initially, robots independently select the task and coalition that maximize their utility. They then share their assignments with neighboring robots within communication range. Conflicts may arise due to the independent assignment; if discrepancies are detected, a robot re-evaluates its choice in the next iteration. This process repeats until all robots are satisfied, leading to a Nash-stable coalition partition where no robot has an incentive to change its assignment. To optimize resource utilization, the algorithm refines the found coalitions by ranking robots based on their utility contributions for each task and only keeping the most suitable robots, up to the number required by the task, to avoid unnecessary costs. The authors of [11] test their method on an earthquake disaster response scenario with tasks like fighting fires and removing landslides requiring large coalitions with 15 robots per task (*UAV*, *UGV*, *USV*) out of fleets with 90 to 240 robots. The average number of algorithm iterations increases linearly with larger team sizes but decreases with more tasks due to a lower probability of conflicting assignments. When reducing the communication range between robots (from 800m down to 50m, with tasks spread out over a 500m × 500m area), the algorithm still achieves 70% of the performance compared to a fully connected network, demonstrating robustness. Limited communication distances lead to fewer iterations because small, disconnected robot groups make decisions internally with reduced conflicts. At moderate distances, the network is strongly but not fully connected, leading to more iterations due to increased conflicts. When communication is sufficient for a fully connected network, efficient information exchange among all robots reduces the number of iterations. The authors do not compare against other methods nor report on runtime analyses or used hardware.

Population-based

In [21], Muhuri et al. compare two extensions of the SGA (Standard *GA*) for solving heterogeneous *MRTA* with coalition formation. The two main operators in *GA* are crossover and mutation, visualized in Figures 4.14 and 4.13. They use a simple chromosome representation, each gene encoding one robot-task assignment. The main problem with SGA is its early convergence to local optima because diversity is eliminated too quickly. To mitigate that, the authors introduce two immigrant-based algorithms:

1. *RIGA (Random Immigrants GA):* Maintains diversity by replacing a portion of the worst individuals in the population with randomly generated individuals in each generation.
2. *EIGA (Elitism Based Immigrants GA):* Builds on the concept of RIGA but generates new immigrants by mutating the best individual (elite) from the previous generation instead of generating random ones.

To further mitigate premature convergence to local optima, the authors introduce adaptive variants (aRIGA and aEIGA) with dynamic mutation and crossover probabilities. These probabilities are calculated based on the gap between the maximum, average, and individual fitness values of the chromosome, ensuring higher mutation for less fit individuals. The fitness function evaluates the value (profit - cost) of scheduling coalitions to tasks, ensuring all required skills are satisfied. If a coalition fails to meet task requirements, the value of scheduling the coalition is set to zero. In their experiments, the authors use 10 robots and 4 tasks, aiming to optimize resource utilization. They provide cost and profit values for robot-task pairs but do not specify how these values were derived. For the most important metric - best solution found per generation - SGA is outperformed significantly by both RIGA and EIGA. The best overall performance is achieved by aEIGA with 95% resource utilization, followed by aRIGA with about 87 % and EIGA with about 85 %. The ”vanilla” SGA only achieves about 60%. This showcases, that both the immigration-based techniques and the adaptive mutation/crossover probabilities enhance performance. For runtime experiments, report SGA to be faster than both RIGA and EIGA (1.07s vs. 2.85s vs. 2.60s), and the adaptive versions prolong the computation slightly. However, they do not report, which hardware was used for the experiments.

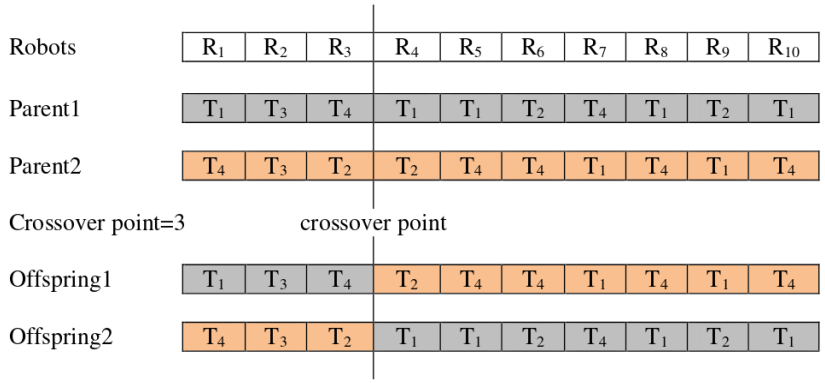

Figure 4.13: Visualization of crossover in *GA* from [21]. A random crossover point is chosen and two new offspring are generated, by switching the two parent segments right of the crossover point.

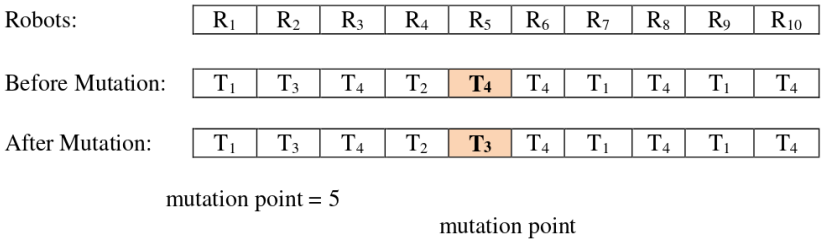

Figure 4.14: Visualization of mutation in *GA* from [21]. Mutation changes the task allocation from one robot to another. Each gene in the chromosome mutates with a given probability. A random mutation point is selected, and the task allocation is updated.

Babincsak et al. are the first to solve the heterogeneous *MRTA* with coalition formation using *ACO* in [19], a metaheuristic inspired by ant behavior - laying and following pheromone trails to find efficient paths to resources [85]. They present two different algorithms, that use the same initialization (greedy solution with deadlock reversal step) and pheromone update rules (evaporation + update based on performance), but differ in their search solution construction:

1. *Territorial ACO (TACO)* which employs an ant-as-robot formulation, each ant representing a single robot. A single ant incrementally selects unassigned tasks based on pheromone levels and heuristic information. Each robot's schedule implicitly defines coalitions.
2. *Swarm Ant System (SAS)* where a single ant represents the entire robot team. A single ant generates a full solution and coalitions are explicitly assigned during solution construction based on the distances from robots to tasks and their potential contributions.

The objective function combines the total cost and the maximum cost for any single robot. Across all test cases, SAS consistently achieves a lower total cost, while TACO achieves a lower maximum cost per robot. This aligns intuitively with the design of each algorithm: in SAS, each ant represents the entire team, while in TACO, each ant represents an individual robot. SAS outperforms TACO in runtime across all scales and swarm sizes. For large problem instances involving 64 tasks, SAS delivers solutions within 200 seconds using high-end hardware (AMD EPYC 7543 processors, 22 CPU cores, and 156 GB of RAM). Although the authors do not provide exact results for smaller problems, their graphs indicate that both TACO and SAS complete tasks with 8 or 16 tasks in single-digit seconds. Both algorithms achieve optimality gaps below 20% compared to the exact *MILP* solution in most scenarios.

In [55], Liu et al. propose a strength learning *PSO* method called SLPSO to optimize multiple objectives (makespan, energy consumption and workload balance) for *MRTA* with heterogeneous robots and dynamic coalitions. In particle swarm optimization, every particle i maintains a position $\mathbf{X_i}$ (in SLPSO $[\mathbf{X}_i^T, \mathbf{X}_i^R]$) and velocity $\mathbf{V_i}$ (in SLPSO $[\mathbf{V}_i^T, \mathbf{V}_i^R]$) and moves towards the global swarm's best solution $\mathbf{g_{best}}$ and the previous best solution of that particle $\mathbf{p_{best_i}}$ [86]. SLPSO initializes the swarm randomly and at the end of each iteration, so-called "spare solutions", which are non-pareto-dominated are stored in an archive. What makes SLPSO special, is the strength learning strategy - Instead of relying solely on global/local best particles, it selects a particle's "strength objective" based on which part of the objective function the particle performs best on. The strategy updates velocity using the particle's personal best and a selected solution from an archive, that performs even better on the chosen "strength objective". The approach has two parts:

1. *Task Permutation* $\mathbf{X_i^T}$: Determines the order of tasks represented by an element-based representation for the set of adjacent tasks. The corresponding velocity $\mathbf{V_i^T}$, indicates the probability of a task to be adjacent to the other tasks. During solution construction, the next task is selected based on these probabilities.
2. *Coalition Permutation* $\mathbf{X_i^R}$: Indicates the coalition for each task with a binary value for each robot, resulting in a binary representation for each task that includes as many elements as there are robots. The velocity $\mathbf{V_i^R}$ is used to guide the assignment of robots to coalitions.

SLPSO addresses convergence and diversity, typical problems for population-based combinatorial optimization problems [86], with heuristic information aiding particle updates for better convergence, and local search enhancing diversity by exploring neighboring regions. The local search is performed on the spare solutions in the archive that have a long distance to neighbors in the objective space to enhance swarm diversity. The local search randomly removes one task from the task permutation, re-inserts it at a different position, and changes one robot coalition (either adding, removing, or replacing a robot).

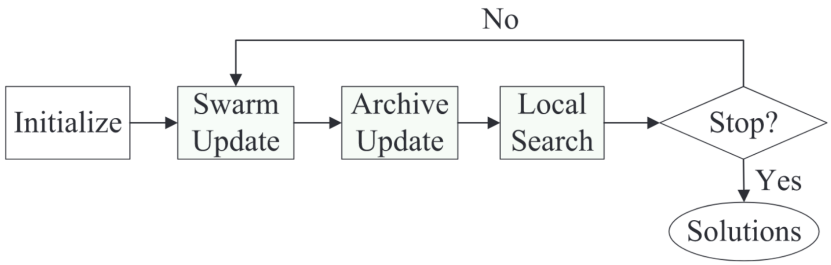

Figure 4.15: Flowchart of SLPSO from [55], extending standard *PSO* with an archive to enable the strength learning and adding a local search step.

The authors do not report the clock runtime, but provide the worst-case complexity of their algorithm (m: number of tasks, n: number of robots, M: number of objectives, N: swarm size) as:

$$O\left(m^2N + mn^3N + MN^2 + m^2n^2M\right)$$

To test SLPSO, the authors use 30 test instances with randomly located tasks (2-5 robot types, 5-20 task types, 5-20 robots, 100-300 tasks,) and use the IGD (distance to Pareto front) and the hypervolume covered for all objective dimensions. They prove that for all test instances, the use of the local search and strength learning heuristic improves performance. They also benchmark against a multiobjective *ACO* method [87], multiobjective *PSO* [88] and a strength-based evolutionary algorithm [89], which they outperform for the majority of test instances for both IGD and hypervolume.

4.4. Table overview of analyzed Methods

Table 4.2 summarizes the analysis of 20 *MRTA* methods with Table 4.3 providing context for cells which require a more nuanced description. Explanation of criteria:

1. *Paper:* Reference to the paper with link to the paragraph that analyzes it (↷ symbol).
2. *Year:* Publication year of paper.
3. *iTax:* Classification based on the taxonomy for *MRTA* problems by Korsah et al. [4].
4. *Heterogeneous:* Does the method model heterogeneity according to the definition in Section 1.1?
5. *Coalition Formation:* Does the method model dynamic coalition formation according to the definition in Section 1.1?
6. *Uncertainty:* Is there any form of probabilistic uncertainty modeled in the method?
7. *Real-time:* Is the method expected to run with approximately 1 Hz on problems similar to the one described in the problem statement in Section 1.4, on a consumer-grade laptop?
   Note: Specifications on the laptop available to the author can be found in Section 1.4.
8. *Centralized vs. Decentralized*: Is the schedule computed on a centralized unit or does each robot assign its tasks autonomously? Centralized approaches generate coordinated plans for all robots, but face challenges in scenarios with degraded communication. Decentralized methods reduce computational load and reliance on central units, but focus on local objectives and can't directly optimize global goals [83].
9. *Task Constraints:* Which constraints can be modeled for tasks? For example precedence (one task must finish before another starts), priority levels (higher-priority tasks are completed earlier if possible), time windows (tasks must be completed within a specific timeframe), and deadlines (tasks must be completed by a certain point).

For the full analysis of the papers, click the **BLACK ARROW (↷)** to jump to the corresponding section.

Paper	Year	Principle	iTax	Heterogeneous	Dyn. Coalitions	Uncertainty	Centralized	Real-time	Task Constraints
[13] ↷	2018	*MILP* Allocator + polynomial Sequencer	*ST-SR-TA-ID*	✓	✗	✓ $U2$	✓	✓	✓ $T4$
[14] ↷	2024	Auction-based + refinement step	*ST-MR-TA-XD*	✓	✗/✓ $S3$	✗	✗	✓	✗
[51] ↷	2022	*IL* + *GAN*	*ST-SR-TA-ID*	✓	✗	✗	✓	✗	✓ $T4$
[52] ↷	2022	Decentralized *RL* + *GCAPCN*	*ST-SR-TA-ID*	✓	✗	✗	✗	✓	✓ $T5$
[53] ↷	2022	Centralized *RL* + *LSTM* schedule propagator	*ST-SR-TA-ID*	✓	✗	✓ $U4$	✓	✗	✓ $T6$
[61] ↷	2023	GNN guiding *ACO* pheromone update	*ST-MR-TA-XD*	✓	✗	✗	✓	✗ $R2$	✓ $T2$

Continued on next page...

Paper	Year	Principle	iTax	Heterogeneous	Dyn. Coalitions	Uncertainty	Centralized	Real-time	Task Constraints
[49]	2023	*GA*	*MT-SR-TA-ND*	✓	✗	✗	✓	✓	✗
[24]	2021	*GA*	*ST-MR-TA-XD*	✗	✓	✗	✓	n.a.^{R5}	✗
[56]	2022	Centralized *RL*	*ST-MR-TA-XD*	✗	✗/✓ ^{S5}	✓^{U3}	✓	✓	✓^{T1}
[9]	2024	Decentralized *RL*	*ST-MR-TA-XD*	✗	✓	✗	✗^{C1}	✓	✗
[50]	2023	Exact *MILP* and greedy version	*ST-MR-TA-XD*	✓	✓	✓^{U1}	✓	✗/✓ ^{R3}	✗
[20]	2021	Stochastic *MIP* + Flow Decomposition	*ST-MR-TA-CD*	✓	✓	✓^{U5}	✓	✗	✓^{T1}
[15]	2020	Construction + improvement heuristic	*ST-MR-TA-XD*	✓	✓	✗	✓	✓^{R4}	✓^{T1}
[48]	2023	*IL*, *GAN* (tasks), *MLP* (robots)	*ST-MR-TA-XD*	✓	✓^{S1}	✗	✓	✓	✓^{T1}
[47]	2024	*IL*, *GATN* (robots+tasks)	*ST-MR-TA-XD*	✓	✓^{S2}	✗	✓	✓	✓^{T1}
[21]	2017	Immigrant-based adaptive *GA*	*ST-MR-IA-ND*	✓	✓	✗	✓	✓	✗
[19]	2023	*ACO*	*ST-MR-TA-XD*	✓	✓	✗	✓	✗^{R1}	✗
[55]	2023	Strength-learning *PSO*	*ST-MR-TA-XD*	✓	✓	✗	✓	✗/✓ ^{R6}	✓^{T1}
[54]	2016	Auction-based + swapping coalitions	*ST-MR-IA-ND*	✓	✓^{S4}	✗	✗	✓	✗/✓ ^{T3}
[11]	2024	Distributed Hedonic Game	*ST-MR-IA-ND*	✓	✓	✗	✗^{C2}	n.a.^{R5}	✗

Table 4.2: Comparison Table for the 20 *MRTA* methods that were analyzed in detail. Table 4.3 provides detailed explanations.

Dynamic Coalitions	
S1	Restricted coalition formation - only happens if a task cannot be completed by a single robot
S2	Advanced coalition formation - with voluntary waiting to enable better coalitions
S3	Coalition formation may occur if two robots happen to work on the same task simultaneously, but it is not explicitly enforced
S4	Coalitions can even change during the execution of a task in case of changes in task requirements or robot capabilities
S5	Coalitions are scheduled to task groups but do not have to be present simultaneously
Task Constraints	
T1	Task precedence constraints possible
T2	Task priority levels possible
T3	Task precedence constraints can be indirectly modeled by only announcing tasks after completion of predecessor
T4	Advanced temporospatial constraints: Wait, Deadlines, two robots cannot be at the same task location simultaneously
T5	Task deadline constraints possible
T6	Task deadline and wait constraints possible
Uncertainty Modeling	
U1	Uncertainty in travel times between tasks
U2	Uncertainty in execution times and scheduled tasks have time windows which allows for robustness against small disturbances in task durations
U3	Uncertainty in task execution times
U4	Uncertainty in task execution times of humans with a Learning curve estimator, taking time-varying proficiencies into account, robots are modeled in a deterministic way
U5	Uncertainty in both robot capabilities and task requirements
Centralized	
C1	Decentralized policy execution, but all agents share the same policy network and training takes place centralized
C2	Distributed Task allocation, with ability to operate in limited-communication scenarios
Real-Time	
R1	Small-sized problems are reported to be solved in real-time, but on high-end hardware
R2	Authors do not go into detail but report ”high” computational cost
R3	Exact *MILP* solution not real-time, but optimal. Greedy solution is real-time
R4	Construction is fast, improvement heuristic offers anytime solution
R5	Authors do not report on runtimes, time complexity or used hardware
R6	Authors do not report on runtimes, but time complexity (↱ for details)

Table 4.3: Detailed explanation of cells from the overview in Table 4.2.

5 Datasets and Benchmarks

This section highlights the lack of standardized benchmarks for heterogeneous *MRTA* with dynamic coalition formation and explores the resulting challenges in objectively comparing different methods.

5.1. Lack of Benchmark Dataset for Use Case

There is no widely adopted benchmark dataset for evaluating *MRTA* methods in general and especially for problems involving heterogeneous robots and dynamic coalition formation, like the one described in Section 1.4. This is evidenced by survey papers on the topic, which do not mention standardized benchmarks and instead highlight the diverse criteria used for comparison [10, 90]. For other research fields with well-established benchmarks, like 3D Object Detection, survey papers do mention them and use these benchmark datasets to draw objective comparisons between methods [91, 92]. For the related methods - *VRP* and *JSSP* - there exist benchmark datasets [93, 94], but as discussed in Section 1.5, these methods cannot be directly applied to the use case or only under the assumptions that task execution time is negligible and no coalitions need to be formed dynamically. Furthermore, Rizk et al. mention that evaluation standards to compare *MRTA* methods are lacking [2], Quinton et al. emphasize the absence of reproducibility and benchmarking efforts [18], and Babincsak et. al state that there is no common benchmark for MRTA with coalition formation [19]. This is underscored by nearly all papers creating problem-specific datasets, as highlighted in Section 5.2.

5.2. Lack of Comparability between Methods

MRTA is a broad, multifaceted field and can be used to tackle a wide variety of problems. The high number of diverse, specialized approaches - each tailored to specific use cases, datasets, solution approaches, modeling assumptions, objective functions, hardware configurations, and reported performance metrics - combined with the lack of a widely adopted benchmark dataset makes a truly objective comparison across methods hard. Although sometimes related methods are compared - like in [51] between ScheduleNet and TERCIO [13] or the method in [9] against CTAS-D [20] - there is no comparison across many methods on the same problems. Additionally, attempts to test on another paper's dataset, as seen in [52] using the dataset from [69], are rare and have yet to establish any dataset as a widely accepted benchmark. The following section provides an overview of some of the factors that contribute to the lack of comparability between methods.

Use case

First of all, the intended use cases are numerous and varied, encompassing areas such as earthquake disaster response scenarios [11], measurement collection in industrial areas [49], production assembly processes [13], or search and rescue missions [14]. Each of these applications has unique requirements and constraints, influencing the design and evaluation of the proposed *MRTA* methods.

Team size

Some studies focus on smaller teams with fewer than ten robots [24, 49, 52], while others address scheduling for large fleets comprising hundreds of robots, forming coalitions with dozens of members

[11]. Additionally, several papers aim to evaluate performance across a broader range, such as [13], which examines teams of 5 to 100 robots, and [9], which explores team sizes from 10 to 40.

Task number
The analyzed literature exhibits significant variation in the number of tasks considered, with some studies focusing on smaller task sets, such as [21, 61], which use 10 tasks or less. Mid-range task sets are explored in works like [24, 49, 53] with 10 to 50 tasks. On the other hand, large-scale scenarios are tackled in studies like [52] using 100 tasks, [9] which explores up to 200 tasks, [55] using 100 to 300, or [13] which tests up to 1024 tasks.

Modeling assumptions
Moreover, there are significant variations in modeling assumptions related to robots, tasks, and coalition formation, as detailed in Section 3. Some methods use homogeneous agents [9, 24, 56], with many more not analyzed in this review. Other Methods use heterogeneous agents, i.e. [14, 19, 47] , with heterogeneity modeled in different ways, as discussed in Section 3.1. Temporal constraints add another layer of complexity. While some methods do not use task constraints at all [11, 21, 95], others incorporate constraints such as task precedence [15, 48, 50], deadlines [13, 51, 52], or wait times [13, 51]. The inclusion or exclusion of these constraints significantly affects the applicability and performance of task allocation methods in different scenarios. For a full overview of all modeling assumptions of analyzed methods, see Section 3 or the overview Table in Section 4.4.

Solution approaches
Solution algorithms in *MRTA* are equally diverse. They range from exact mathematical formulations —such as *MILP* [50] - to heuristic and metaheuristic methods like genetic algorithms [21, 24, 49], and swarm-based methods [19, 55]. Auction-based mechanisms [14, 54] are popular for their scalability and distributed nature, while learning-based methods using graphs and *GNN* — trained by Reinforcement Learning [9, 52, 56] or Imitation Learning [47, 48, 51] — have recently found a lot of attention. For a detailed description of all analyzed solution methods and their underlying approach see Chapter 4.

Objective Functions
Some of the objective functions used include minimizing energy consumption [49], overall makespan [9, 47, 48, 50, 51], makespan with other custom extensions [13], average completion time of task groups [56], or maximizing number of completed tasks within their given deadline [52]. Some papers also use multiple objectives, like [55] that optimize for makespan, workload balance and energy consumption at the same time or [20] which consider makespan, energy consumption, and risk to create a robust approach. In their *MRTA* survey paper [10], Chakraa et al. additionally name minimizing travel distance and waiting times as objectives that some studies focus on.

Performance Metrics
The evaluation of these methods often lacks standardization. Performance metrics that are used for evaluation include optimality gap [14, 47] compared to the exact solution, percentage of problems solved within optimality w.r.t exact solution [51], runtime or execution frequency [9, 15, 47, 49, 56], idle rate of the robots [48], robustness in terms of differing constraints [13], hypervolume covered in the multi-objective space [55], average number of iterations for decentralized auctions [11], percentage of feasibility of generated schedules [53] or task completion within deadlines [52] — all of which are not consistently reported across studies.

Used Hardware and Runtime Performance
As heterogeneous *MRTA* with dynamic coalition formation is proven to be NP-hard [84] and finding the exact solution is exponential in both the number of tasks and robots [96], runtime in combination with used hardware is a crucial factor in an *MRTA* algorithm's performance. However, not all studies report their runtime performance or provide an analysis of algorithmic complexity [11, 21, 24, 52, 55]. For the papers that do offer these metrics, the diversity in hardware platforms — combined with differences in problem types and sizes — makes objective runtime comparisons impossible. Some researchers utilize high-performance hardware to obtain their results [19, 51], while others rely on standard consumer-grade hardware with significantly lower computational capabilities [15, 49].

Combinatorial Compounding
As shown above, approaches vary significantly across multiple factors, with differences compounding across axes, creating a combinatorial explosion that complicates direct comparisons.

6

Conclusion and Future Directions

MRTA plays an important role in enabling fully autonomous robot systems in the future. Incorporating dynamic coalition formation with heterogeneous robots and real-time re-planning that respects precedence constraints can significantly enhance the deployability and cost-efficiency of these systems. *MRTA* has achieved significant attention from the academic community, but research is highly diverse and hard to compare. This chapter provides a summary of the key insights from this literature review and identifies future research opportunities to advance *MRTA*. It begins by examining the current state of the field, including common modeling assumptions and a review of existing conventional and learning-based solution methods. It then discusses challenges in comparing and evaluating methods and concludes with specific recommendations for future research directions.

Diverse modeling assumptions in the literature highlight the complexity of balancing model simplicity with the level of detail needed to deploy the algorithms on real robots, as detailed in Chapter 3. Heterogeneity is modeled using approaches ranging from simple binary capability models to more advanced formulations with execution quality or stochastic capability variations. Tasks are categorized as either atomic, requiring all coalition members to execute simultaneously, or non-atomic, allowing incremental execution as robots arrive. Different temporal constraints on tasks are considered with precedence constraints being the most common. To account for uncertainty due to imperfect information in real-world scenarios, some methods model execution and travel times as stochastic variables. Importantly, most papers assume the task set to be well-defined, known, and decomposed, which does not necessarily hold true in real-life autonomous systems. Another common assumption of *MRTA* methods is a reliable communication network to which all robots are connected. For an algorithm to work in real scenarios, robustness against communication failures would prove beneficial.

To address *MRTA* with heterogeneous robots and coalition formation, several types of methods could be considered. Conventional approaches include slow but exact *MILP* solutions and faster yet suboptimal greedy algorithms. Other methods, such as population-based optimization techniques (e.g. *GA* or *ACO*), leverage iterative search processes over a population of potential solutions, balancing exploration and exploitation. Auction-based methods provide scalability and decentralized task allocation by having robots bid for tasks. For a full overview of potential solution methods, refer to Chapter 4.

In recent years, learning-based methods have gained increasing attention, particularly those that combine graph-based encoders (*GNN*, *GAN*, *GATN*, *GCAPCN*) with attention-based decoders. These methods, trained via *RL* or *IL* offer the potential for fast runtimes, good performance, and scalability. *RL*-based methods typically model the problem as an *MDP*, where actions involve appending tasks to robots' schedules. While *RL* does not require ground-truth solutions, training can be inefficient due to time spent exploring infeasible solution permutations, especially in scenarios with temporal constraints and heterogeneous robots. *IL*-based methods, on the other hand, often frame the problem as bipartite graph matching, with the neural network estimating the assignment reward matrix between robots and tasks. However, *IL* requires ground-truth solutions from an exact solver for training, which can be computationally expensive to generate. These exact solutions then serve as demonstrations to be imitated

by the *IL* process. Most existing *RL* methods address either coalition formation with homogeneous robots or task allocation for heterogeneous robots without coalition formation. The few learning-based approaches that address both heterogeneous robots and dynamic coalition formation primarily rely on *IL* with bipartite matching, as illustrated in Section 4.3.

Objective comparison between *MRTA* methods is challenging due to the highly diverse nature of approaches and the lack of an accepted benchmark dataset or standardized evaluation framework, as outlined in Chapter 5. Each study typically creates a dataset tailored to its specific problem. Additionally, the diversity in use cases, team sizes, task numbers, modeling assumptions, underlying algorithmic approaches, and objectives further complicates the evaluation process. The type and focus of reported results vary significantly across papers, often using different performance metrics and hardware, making it challenging to assess methods in a standardized manner.

Future research could explore the following aspects:

1. Prioritize learning-based methods as they hold the biggest promise in performance gains to advance the state of the art in *MRTA*, given the recent progress in machine learning and AI. A deeper understanding of the contributions of each system component is essential. For instance, studies comparing different encoder structures, such as *GAN* or *GCAPCN*, in combination with transformer-based architectures, could provide valuable insights. Additionally, research should explore the influence of various training algorithms, comparing *IL* and *RL* across different problem classes to offer clear recommendations. Investigating the scalability of these networks relative to problem complexity and size also plays a key role.
2. Focus on bridging the gap between perception and reasoning systems, generating and decomposing tasks, and the scheduling algorithms that assign robots to these tasks. A seamless integration would enable true autonomy by decreasing reliance on manually specifying tasks and enhance the adaptability and efficiency of robot teams in dynamic, real-world environments.
3. Develop more robust MRTA solutions tailored for real-world deployment, considering uncertainties in travel times, execution durations, and task success rates. This would account for the fact that complex motions like grasping or manipulating objects in real scenarios are not yet reliable enough to assume success rates of 100% or deterministic execution times.
4. Research the integration of learning-based methods with conventional optimization techniques. Guiding population-based optimization with information derived from graph-based encoders like informing the *ACO* pheromone update or choosing the next node to branch on in branch-and-bound solvers could significantly speed up computation.
5. Developing a standardized benchmark dataset is essential. This dataset should cover a wide range of problem sizes, incorporating both heterogeneous and homogeneous robots, coalition formation scenarios, and various temporal constraints. Including simpler scenarios (e.g. homogeneous robots) in the same dataset format would allow complex approaches to be benchmarked against simpler, existing techniques on the use cases the simpler methods can handle. It should also include ground-truth solutions optimized for different objective functions, as well as standardized train-test splits. Such a dataset would facilitate objective comparisons between methods.

To summarize, the *MRTA* field would benefit from standardized benchmarks and datasets. Furthermore, learning-based algorithms show great potential due to their promising trade-off between performance and runtime. Therefore, it could be beneficial to push learning-based methods to handle the full complexity of heterogeneous robot teams and dynamically formed coalitions in real-world scenarios with temporal constraints and real-time requirements.

Therefore, the subsequent thesis will focus on implementing learning-based methods to gain a deeper understanding of the limitations, potential, and best practices of those methods (Future research 1). Additionally, a standardized dataset for objectively benchmarking different methods will be created (Future research 5). For a more detailed research plan refer to Chapter 7.

7

Research Plan

This chapter outlines the core contributions, baseline methods to compare with, and implementation timeline for the thesis. It is important to note that this chapter is preliminary, as the final decision on the dataset and methods has not yet been made.

7.1. Planned Contributions

The main contributions that the thesis aims to provide can be summarized as:

1. **Develop learning-based Scheduling Solution**

   Provide a full solution to schedule heterogeneous robots with dynamic coalitions and precedence constraints in real-time for the use case described in Section 1.4. Conduct experiments like ablation studies or architectural variations to gain deeper insights into the system's performance and potential improvements. Additionally, implement the solution on physical robots from the Autonomous Multi-Robots Lab[1] to validate its practicality.

2. **Dataset with optimal Solutions**

   Generate a dataset inspired by the robotic bar use case described in Section 1.4 with varying team sizes, task and skill numbers, precedence constraints, and difficulties. Solve each instance optimally to establish ground truth for learning-based approaches and benchmarking of solutions and baselines. Provide a train-test split to facilitate consistent evaluation of learning-based methods and allow other researchers to evaluate their solutions objectively.

3. **Adapted Baseline Implementations for Dataset**

   Implement the chosen baseline methods from Section 7.2 and apply them to the generated dataset. Include these implementations in the dataset repository to provide a standardized reference for comparing other methods. Conduct comprehensive experiments to compare baseline performance with the developed solution across key metrics such as solution quality (makespan and extensions), runtime, and scalability.

For further details on the preliminary timeline see Figure 7.1.

[1] https://autonomousrobots.nl

7.2. Baselines

This chapter outlines potential baseline methods for comparison with the developed *MRTA* solution using the created dataset. Given the limited availability of public code implementations, learning-based methods will only be considered as baselines if code is publicly available. For conventional methods, re-implementation from scratch is feasible.

Core Baselines

Both an *MILP* formulation and a greedy heuristic for heterogeneous *MRTA* with coalition formation will be implemented. Aswale et al. [50] provide an exact *MILP* solution for tasks with binary agent heterogeneity and dynamic coalition formation. They also propose a greedy heuristic that aims to balance coalition size (by minimizing redundant skills) with travel time to reduce makespan. Both methods will be extended to incorporate precedence constraints to handle scenarios similar to those described in Section 1.4. The *MILP* solution will serve as a benchmark for optimality and can generate ground-truth solutions for training learning-based methods. The greedy heuristic offers extremely fast computation times, but at the cost of reduced performance. Therefore, it can serve as a baseline for real-time methods that should achieve better performance than the simple greedy heuristic.

Potential Baselines

The following methods are preliminary potential baselines, as the final selection will depend on the specifics of the dataset and the developed scheduling solution. Given the limited number of methods that handle both heterogeneity and coalition formation while respecting precedence constraints, it may be valuable to benchmark on several simplified scenarios that other methods can handle, such as coalition formation with homogeneous robots.

Potential baselines include:

- The *RL* method by Dai et al. [9] that tackles dynamic coalition formation of homogeneous robots. It does not support precedence constraints or heterogeneity but offers a code implementation[2]. Analyzed here.
- CapAM, a *RL* solution by Paul et al. [52] dealing with mildly heterogeneous robots (different execution speeds, but every robot can perform any task). It does not form dynamic coalitions but can model deadline constraints and has an available implementation[3]. Analyzed here.
- CTAS by Fu et al. [20] that uses stochastic *MIP* and Flow Decomposition, modeling heterogeneous MRTA with dynamic coalitions, accounting for uncertainty in robot capabilities and task requirements, with well-documented implementation available[4]. Analyzed here.
- Implementing one of the *ACO* algorithm variants TACO and SAS from [19]. These methods can deal with heterogeneous *MRTA* problems with coalition formation, but not with temporal constraints and no implementation is publicly available. Analyzed here.
- A *GA* implementation from scratch, inspired by the method [49] (analyzed here) for heterogeneous robots or the method [24] using a *GA* for dynamic coalition formation with homogeneous robots (analyzed here). Potential performance increases could be achieved by incorporating dynamic mutation and crossover probabilities like in [21] (analyzed here).
- CoLoSSI, the auction-based method by Ansari et. al [14], modeling non-atomic tasks for heterogeneous robots without coalition formation and unable to handle temporal task constraints. Implementation is available[5]. Analyzed here.

[2] https://github.com/marmotlab/DCMRTA/tree/main
[3] https://github.com/adamslab-ub/CapAM-MRTA
[4] https://gitlab.com/barton-research-group/open/resilient_team_planner
[5] https://github.com/IshaqAnsari2001/CoLoSSI

7.3. Timeline

Figure 7.1 shows the preliminary timeline for the thesis project. The project officially started on October 15, 2024, and the intended finish date is the end of July 2025.

The process can be divided into Writing and Implementation work:

Writing

- *Literature Review:* Thorough review of existing literature and solution methods.
- *Thesis Report:* Reporting on developed method, experiments and comparison against baselines.
- *Thesis Defence:* End presentation to attain the MSc degree.

Implementation

- *Initial Tests + Exact MILP solution:* Implement both the *MILP* exact method and the greedy heuristic from [50], scripts to generate example problems, visualize resulting schedules and comparison of runtime and makespan.
- *Formalize own Method + Dataset:* Preliminary detailed formulation of the solution approach and the corresponding dataset needs resulting from it.
- *Dataset Creation:* Generate the dataset with corresponding optimal solutions.
- *Baselines:* Implementation of the baselines chosen corresponding to Section 7.2.
- *Own Method:* Create and implement own method to solve the problem defined in Section 1.4.
- *Experiments + Ablations:* Comparison with baselines, conduct experiments like ablation studies.
- *Real-life Tests:* Implementation on actual robots in the Autonomous Multi-Robots Lab [6].

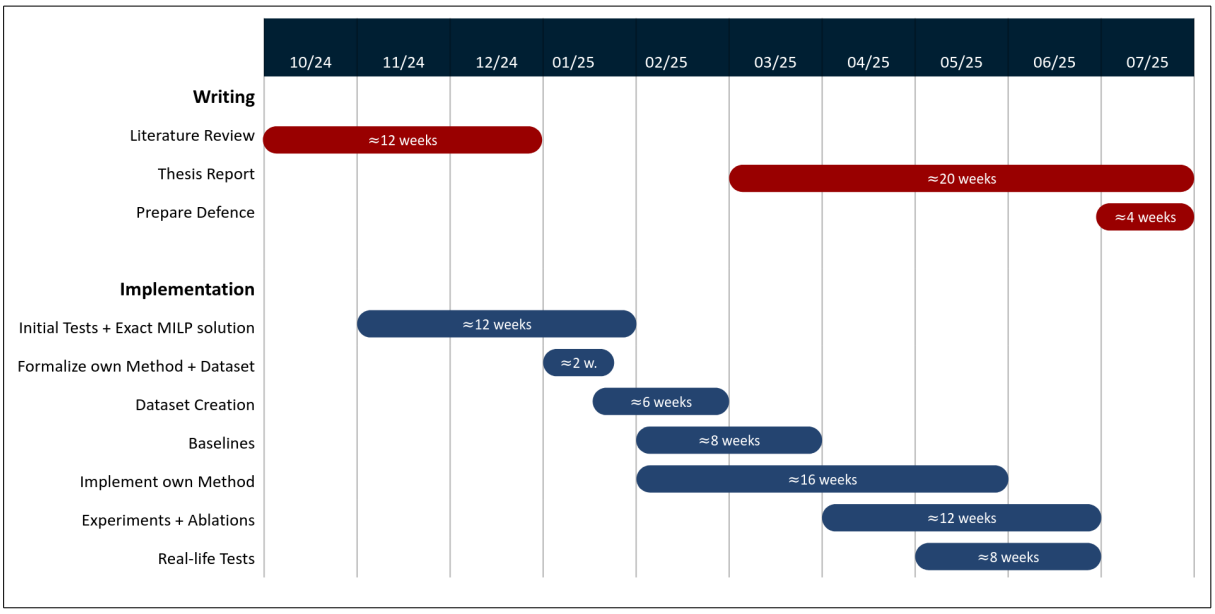

Figure 7.1: Preliminary Timeline for the thesis

[6] https://autonomousrobots.nl

References

[1] Chayan Sarkar, Himadri Sekhar Paul, and Arindam Pal. “A Scalable Multi-Robot Task Allocation Algorithm”. In: *2018 IEEE International Conference on Robotics and Automation (ICRA)*. ISSN: 2577-087X. May 2018, pp. 5022–5027. DOI: 10.1109/ICRA.2018.8460886. URL: https://ieeexplore.ieee.org/document/8460886 (visited on 10/20/2024).

[2] Yara Rizk, Mariette Awad, and Edward W. Tunstel. “Cooperative Heterogeneous Multi-Robot Systems: A Survey”. en. In: *ACM Computing Surveys* 52.2 (Mar. 2020), pp. 1–31. ISSN: 0360-0300, 1557-7341. DOI: 10.1145/3303848. URL: https://dl.acm.org/doi/10.1145/3303848 (visited on 10/17/2024).

[3] Haris Aziz et al. *Task Allocation using a Team of Robots*. en. arXiv:2207.09650 [cs]. July 2022. URL: http://arxiv.org/abs/2207.09650 (visited on 10/14/2024).

[4] G. Ayorkor Korsah, Anthony Stentz, and M. Bernardine Dias. “A comprehensive taxonomy for multi-robot task allocation”. en. In: *The International Journal of Robotics Research* 32.12 (Oct. 2013), pp. 1495–1512. ISSN: 0278-3649, 1741-3176. DOI: 10.1177/0278364913496484. URL: https://journals.sagepub.com/doi/10.1177/0278364913496484 (visited on 10/16/2024).

[5] Brian P. Gerkey and Maja J. Matarić. “A Formal Analysis and Taxonomy of Task Allocation in Multi-Robot Systems”. en. In: *The International Journal of Robotics Research* 23.9 (Sept. 2004), pp. 939–954. ISSN: 0278-3649, 1741-3176. DOI: 10.1177/0278364904045564. URL: https://journals.sagepub.com/doi/10.1177/0278364904045564 (visited on 10/15/2024).

[6] A. Farinelli, L. Iocchi, and D. Nardi. “Multirobot systems: a classification focused on coordination”. In: *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)* 34.5 (Oct. 2004). Conference Name: IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics), pp. 2015–2028. ISSN: 1941-0492. DOI: 10.1109/TSMCB.2004.832155. URL: https://ieeexplore.ieee.org/document/1335496 (visited on 10/25/2024).

[7] Matteo Bettini, Ajay Shankar, and Amanda Prorok. *Heterogeneous Multi-Robot Reinforcement Learning*. en. arXiv:2301.07137 [cs]. Jan. 2023. URL: http://arxiv.org/abs/2301.07137 (visited on 10/15/2024).

[8] Chuang Huang, Hao Zhang, and Zhuping Wang. “Task Allocation of Multi-robot Coalition Formation”. en. In: *Proceedings of 2021 5th Chinese Conference on Swarm Intelligence and Cooperative Control*. Ed. by Zhang Ren, Mengyi Wang, and Yongzhao Hua. Singapore: Springer Nature, 2023, pp. 221–230. ISBN: 978-981-19399-8-3. DOI: 10.1007/978-981-19-3998-3_22.

[9] Weiheng Dai, Aditya Bidwai, and Guillaume Sartoretti. “Dynamic Coalition Formation and Routing for Multirobot Task Allocation via Reinforcement Learning”. en. In: *2024 IEEE International Conference on Robotics and Automation (ICRA)*. Yokohama, Japan: IEEE, May 2024, pp. 16567–16573. ISBN: 9798350384574. DOI: 10.1109/ICRA57147.2024.10611244. URL: https://ieeexplore.ieee.org/document/10611244/ (visited on 10/17/2024).

[10] Hamza Chakraa et al. “Optimization techniques for Multi-Robot Task Allocation problems: Review on the state-of-the-art”. en. In: *Robotics and Autonomous Systems* 168 (Oct. 2023), p. 104492. ISSN: 09218890. DOI: 10.1016/j.robot.2023.104492. URL: https://linkinghub.elsevier.com/retrieve/pii/S0921889023001318 (visited on 10/15/2024).

[11] Liwang Zhang et al. “Task Allocation in Heterogeneous Multi-Robot Systems Based on Preference-Driven Hedonic Game”. In: *2024 IEEE International Conference on Robotics and Automation (ICRA)*. May 2024, pp. 8967–8972. DOI: 10.1109/ICRA57147.2024.10611476. URL: https://ieeexplore.ieee.org/document/10611476 (visited on 11/29/2024).

[12] Walker Gosrich et al. *Multi-Robot Coordination and Cooperation with Task Precedence Relationships*. en. arXiv:2209.14417 [cs]. May 2023. DOI: 10.48550/arXiv.2209.14417. URL: http://arxiv.org/abs/2209.14417 (visited on 12/10/2024).

[13] Matthew C. Gombolay, Ronald J. Wilcox, and Julie A. Shah. “Fast Scheduling of Robot Teams Performing Tasks With Temporospatial Constraints”. In: *IEEE Transactions on Robotics* 34.1 (Feb. 2018). Conference Name: IEEE Transactions on Robotics, pp. 220–239. ISSN: 1941-0468. DOI:

10.1109/TRO.2018.2795034. URL: https://ieeexplore.ieee.org/document/8279546 (visited on 11/21/2024).

[14] Ishaq Ansari et al. “CoLoSSI: Multi-Robot Task Allocation in Spatially-Distributed and Communication Restricted Environments”. In: *IEEE Access* 12 (2024). Conference Name: IEEE Access, pp. 132838–132855. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2024.3434645. URL: https://ieeexplore.ieee.org/document/10613777/?arnumber=10613777 (visited on 11/01/2024).

[15] Esther Bischoff et al. *Multi-Robot Task Allocation and Scheduling Considering Cooperative Tasks and Precedence Constraints*. en. arXiv:2005.03902 [eess]. May 2020. URL: http://arxiv.org/abs/2005.03902 (visited on 10/24/2024).

[16] Ragesh K. Ramachandran, James A. Preiss, and Gaurav S. Sukhatme. “Resilience by Reconfiguration: Exploiting Heterogeneity in Robot Teams”. In: *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. ISSN: 2153-0866. Nov. 2019, pp. 6518–6525. DOI: 10.1109/IROS40897.2019.8968611. URL: https://ieeexplore.ieee.org/document/8968611 (visited on 12/10/2024).

[17] Alaa Khamis, Ahmed Hussein, and Ahmed Elmogy. “Multi-robot Task Allocation: A Review of the State-of-the-Art”. en. In: *Cooperative Robots and Sensor Networks 2015*. Ed. by Anis Koubâa and J.Ramiro Martínez-de Dios. Cham: Springer International Publishing, 2015, pp. 31–51. ISBN: 978-3-319-18299-5. DOI: 10.1007/978-3-319-18299-5_2. URL: https://doi.org/10.1007/978-3-319-18299-5_2 (visited on 12/10/2024).

[18] Félix Quinton, Christophe Grand, and Charles Lesire. “Market Approaches to the Multi-Robot Task Allocation Problem: a Survey”. en. In: *Journal of Intelligent & Robotic Systems* 107.2 (Feb. 2023), p. 29. ISSN: 1573-0409. DOI: 10.1007/s10846-022-01803-0. URL: https://doi.org/10.1007/s10846-022-01803-0 (visited on 11/11/2024).

[19] William Babincsak, Ashay Aswale, and Carlo Pinciroli. “Ant Colony Optimization for Heterogeneous Coalition Formation and Scheduling with Multi-Skilled Robots”. In: *2023 International Symposium on Multi-Robot and Multi-Agent Systems (MRS)*. Dec. 2023, pp. 121–127. DOI: 10.1109/MRS60187.2023.10416771. URL: https://ieeexplore.ieee.org/document/10416771 (visited on 10/17/2024).

[20] Bo Fu et al. *Robust Task Scheduling for Heterogeneous Robot Teams under Capability Uncertainty*. June 2021. URL: https://www.researchgate.net/publication/353070348_Robust_Task_Scheduling_for_Heterogeneous_Robot_Teams_under_Capability_Uncertainty (visited on 11/12/2024).

[21] Pranab Muhuri and Amit Rauniyar. “Immigrants Based Adaptive Genetic Algorithms for Task Allocation in Multi-Robot Systems”. In: *International Journal of Computational Intelligence and Applications* 16 (Dec. 2017), p. 1750025. DOI: 10.1142/S1469026817500250.

[22] Ying Tan and Zhong-yang Zheng. “Research Advance in Swarm Robotics”. In: *Defence Technology* 9.1 (Mar. 2013), pp. 18–39. ISSN: 2214-9147. DOI: 10.1016/j.dt.2013.03.001. URL: https://www.sciencedirect.com/science/article/pii/S221491471300024X (visited on 11/04/2024).

[23] Maria Gini. “Multi-Robot Allocation of Tasks with Temporal and Ordering Constraints”. en. In: *Proceedings of the AAAI Conference on Artificial Intelligence* 31.1 (Feb. 2017). ISSN: 2374-3468, 2159-5399. DOI: 10.1609/aaai.v31i1.11145. URL: https://ojs.aaai.org/index.php/AAAI/article/view/11145 (visited on 11/28/2024).

[24] Muhammad Usman Arif. “Robot coalition formation against time-extended multi-robot tasks”. In: *International Journal of Intelligent Unmanned Systems* 10.4 (June 2021). Publisher: Emerald Publishing Limited, pp. 468–481. ISSN: 2049-6427. DOI: 10.1108/IJIUS-12-2020-0070. URL: https://doi.org/10.1108/IJIUS-12-2020-0070 (visited on 10/17/2024).

[25] J. M. Garcia-Haro et al. “Service Robots in Catering Applications: A Review and Future Challenges”. In: *Electronics* (2020). URL: https://api.semanticscholar.org/CorpusID:234451793.

[26] Oliver Bendel and Lea K. Peier. “How Can Bar Robots Enhance the Well-being of Guests?” In: Version Number: 1. arXiv, 2023. DOI: 10.48550/ARXIV.2304.14410. URL: https://arxiv.org/abs/2304.14410 (visited on 12/14/2024).

[27] Paola Ardón et al. *Affordances in Robotic Tasks – A Survey*. en. arXiv:2004.07400 [cs]. Apr. 2020. URL: http://arxiv.org/abs/2004.07400 (visited on 11/12/2024).

[28] G. B. Dantzig and J. H. Ramser. “The Truck Dispatching Problem”. In: *Management Science* 6.1 (1959). Publisher: INFORMS, pp. 80–91. ISSN: 0025-1909. URL: https://www.jstor.org/stable/2627477 (visited on 11/01/2024).

[29] Gilbert Laporte. “The Traveling Salesman Problem, the Vehicle Routing Problem, and Their Impact on Combinatorial Optimization:” en. In: *International Journal of Strategic Decision Sciences* 1.2 (Apr. 2010), pp. 82–92. ISSN: 1947-8569, 1947-8577. DOI: 10.4018/jsds.2010040104. URL: https://services.igi-global.com/resolvedoi/resolve.aspx?doi=10.4018/jsds.2010040104 (visited on 12/16/2024).

[30] A. Mor and M. G. Speranza. “Vehicle routing problems over time: a survey”. en. In: *4OR* 18.2 (June 2020), pp. 129–149. ISSN: 1614-2411. DOI: 10.1007/s10288-020-00433-2. URL: https://doi.org/10.1007/s10288-020-00433-2 (visited on 11/01/2024).

[31] Çağrı Koç et al. “Thirty years of heterogeneous vehicle routing”. In: *European Journal of Operational Research* 249.1 (Feb. 2016), pp. 1–21. ISSN: 0377-2217. DOI: 10.1016/j.ejor.2015.07.020. URL: https://www.sciencedirect.com/science/article/pii/S0377221715006530 (visited on 11/01/2024).

[32] Artur Pessoa, Ruslan Sadykov, and Eduardo Uchoa. “Enhanced Branch-Cut-and-Price algorithm for heterogeneous fleet vehicle routing problems”. In: *European Journal of Operational Research* 270.2 (Oct. 2018), pp. 530–543. ISSN: 0377-2217. DOI: 10.1016/j.ejor.2018.04.009. URL: https://www.sciencedirect.com/science/article/pii/S0377221718303126 (visited on 11/01/2024).

[33] Baozhen Yao et al. “An improved particle swarm optimization for carton heterogeneous vehicle routing problem with a collection depot”. en. In: *Annals of Operations Research* 242.2 (July 2016), pp. 303–320. ISSN: 1572-9338. DOI: 10.1007/s10479-015-1792-x. URL: https://doi.org/10.1007/s10479-015-1792-x (visited on 11/01/2024).

[34] Vitor Coelho et al. “An ILS-based Algorithm to Solve a Large-scale Real Heterogeneous Fleet VRP with Multi-trips and Docking Constraints”. In: *European Journal of Operational Research* 250 (Apr. 2016). DOI: 10.1016/j.ejor.2015.09.047.

[35] Shuguang Liu. “A hybrid population heuristic for the heterogeneous vehicle routing problems”. In: *Transportation Research Part E: Logistics and Transportation Review* 54 (Aug. 2013), pp. 67–78. ISSN: 1366-5545. DOI: 10.1016/j.tre.2013.03.010. URL: https://www.sciencedirect.com/science/article/pii/S1366554513000604 (visited on 11/01/2024).

[36] Mustafa Avci and Seyda Topaloglu. “A hybrid metaheuristic algorithm for heterogeneous vehicle routing problem with simultaneous pickup and delivery”. In: *Expert Systems with Applications* 53 (July 2016), pp. 160–171. ISSN: 0957-4174. DOI: 10.1016/j.eswa.2016.01.038. URL: https://www.sciencedirect.com/science/article/pii/S0957417416000610 (visited on 11/01/2024).

[37] Jun Li et al. “Colored Traveling Salesman Problem and Solution”. In: *IFAC Proceedings Volumes*. 19th IFAC World Congress 47.3 (Jan. 2014), pp. 9575–9580. ISSN: 1474-6670. DOI: 10.3182/20140824-6-ZA-1003.01403. URL: https://www.sciencedirect.com/science/article/pii/S1474667016431289 (visited on 12/16/2024).

[38] Xiaoshan Bai et al. “Group-Based Distributed Auction Algorithms for Multi-Robot Task Assignment”. In: *IEEE Transactions on Automation Science and Engineering* 20.2 (Apr. 2023). Conference Name: IEEE Transactions on Automation Science and Engineering, pp. 1292–1303. ISSN: 1558-3783. DOI: 10.1109/TASE.2022.3175040. URL: https://ieeexplore.ieee.org/document/9781631 (visited on 11/29/2024).

[39] Wei Qin et al. “A novel reinforcement learning-based hyper-heuristic for heterogeneous vehicle routing problem”. In: *Computers & Industrial Engineering* 156 (June 2021), p. 107252. ISSN: 0360-8352. DOI: 10.1016/j.cie.2021.107252. URL: https://www.sciencedirect.com/science/article/pii/S036083522100156X (visited on 10/22/2024).

[40] Armin Sadeghi and Stephen L. Smith. “Heterogeneous Task Allocation and Sequencing via Decentralized Large Neighborhood Search”. en. In: *Unmanned Systems* 05.02 (Apr. 2017), pp. 79–95. ISSN: 2301-3850, 2301-3869. DOI: 10.1142/S2301385017500066. URL: https://www.worldscientific.com/doi/abs/10.1142/S2301385017500066 (visited on 12/03/2024).

[41] Banu Çaliş and Serol Bulkan. “A research survey: review of AI solution strategies of job shop scheduling problem”. en. In: *Journal of Intelligent Manufacturing* 26.5 (Oct. 2015), pp. 961–973. ISSN: 1572-8145. DOI: 10.1007/s10845-013-0837-8. URL: https://doi.org/10.1007/s10845-013-0837-8 (visited on 11/06/2024).

[42] Hegen Xiong et al. “A survey of job shop scheduling problem: The types and models”. In: *Computers & Operations Research* 142 (June 2022), p. 105731. ISSN: 0305-0548. DOI: 10.1016/j.cor.2022.105731. URL: https://www.sciencedirect.com/science/article/pii/S0305054822000338 (visited on 10/22/2024).

[43] Igor G. Smit et al. *Graph Neural Networks for Job Shop Scheduling Problems: A Survey*. en. arXiv:2406.14096 [cs]. June 2024. URL: http://arxiv.org/abs/2406.14096 (visited on 10/13/2024).

[44] Lucas Berterottière, Stéphane Dauzère-Pérès, and Claude Yugma. "Flexible job-shop scheduling with transportation resources". In: *European Journal of Operational Research* 312.3 (Feb. 2024), pp. 890–909. ISSN: 0377-2217. DOI: 10.1016/j.ejor.2023.07.036. URL: https://www.sciencedirect.com/science/article/pii/S0377221723005969 (visited on 11/06/2024).

[45] Imanol Echeverria, Maialen Murua, and Roberto Santana. *Solving the flexible job-shop scheduling problem through an enhanced deep reinforcement learning approach*. arXiv:2310.15706. Jan. 2024. DOI: 10.48550/arXiv.2310.15706. URL: http://arxiv.org/abs/2310.15706 (visited on 10/22/2024).

[46] Longkang Li et al. *Learning to Optimize Permutation Flow Shop Scheduling via Graph-based Imitation Learning*. arXiv:2210.17178. Dec. 2023. DOI: 10.48550/arXiv.2210.17178. URL: http://arxiv.org/abs/2210.17178 (visited on 10/15/2024).

[47] Williard Joshua Jose and Hao Zhang. "Learning for Dynamic Subteaming and Voluntary Waiting in Heterogeneous Multi-Robot Collaborative Scheduling". en. In: *IEEE International Conference on Robotics and Automation (ICRA)* (2024).

[48] Peng Gao et al. "Collaborative Scheduling with Adaptation to Failure for Heterogeneous Robot Teams". In: *2023 IEEE International Conference on Robotics and Automation (ICRA)*. May 2023, pp. 1414–1420. DOI: 10.1109/ICRA48891.2023.10161502. URL: https://ieeexplore.ieee.org/document/10161502/?arnumber=10161502 (visited on 10/14/2024).

[49] Hamza Chakraa et al. "A Centralized Task Allocation Algorithm for a Multi-Robot Inspection Mission With Sensing Specifications". In: *IEEE Access* 11 (2023). Conference Name: IEEE Access, pp. 99935–99949. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2023.3315130. URL: https://ieeexplore.ieee.org/abstract/document/10250421 (visited on 10/15/2024).

[50] Ashay Aswale and Carlo Pinciroli. *Heterogeneous Coalition Formation and Scheduling with Multi-Skilled Robots*. en. arXiv:2306.11936 [cs]. June 2023. URL: http://arxiv.org/abs/2306.11936 (visited on 08/21/2024).

[51] Zheyuan Wang, Chen Liu, and Matthew Gombolay. "Heterogeneous graph attention networks for scalable multi-robot scheduling with temporospatial constraints". en. In: *Autonomous Robots* 46.1 (Jan. 2022), pp. 249–268. ISSN: 0929-5593, 1573-7527. DOI: 10.1007/s10514-021-09997-2. URL: https://link.springer.com/10.1007/s10514-021-09997-2 (visited on 08/13/2024).

[52] Steve Paul, Payam Ghassemi, and Souma Chowdhury. *Learning Scalable Policies over Graphs for Multi-Robot Task Allocation using Capsule Attention Networks*. en. arXiv:2205.03321 [cs]. May 2022. URL: http://arxiv.org/abs/2205.03321 (visited on 10/14/2024).

[53] Batuhan Altundas et al. "Learning Coordination Policies over Heterogeneous Graphs for Human-Robot Teams via Recurrent Neural Schedule Propagation". en. In: *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. arXiv:2301.13279 [cs]. Oct. 2022, pp. 11679–11686. DOI: 10.1109/IROS47612.2022.9981748. URL: http://arxiv.org/abs/2301.13279 (visited on 10/14/2024).

[54] Muhammad Irfan and Adil Farooq. "Auction-based task allocation scheme for dynamic coalition formations in limited robotic swarms with heterogeneous capabilities". In: *2016 International Conference on Intelligent Systems Engineering (ICISE)*. Jan. 2016, pp. 210–215. DOI: 10.1109/IN□SE.2016.7475122. URL: https://ieeexplore.ieee.org/document/7475122 (visited on 10/17/2024).

[55] Xiao-Fang Liu et al. "Strength Learning Particle Swarm Optimization for Multiobjective Multirobot Task Scheduling". In: *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 53.7 (July 2023). Conference Name: IEEE Transactions on Systems, Man, and Cybernetics: Systems, pp. 4052–4063. ISSN: 2168-2232. DOI: 10.1109/TSMC.2023.3239953. URL: https://ieeexplore.ieee.org/document/10041025/?arnumber=10041025 (visited on 10/24/2024).

[56] Fuqin Deng et al. "A Learning Approach to Multi-robot Task Allocation with Priority Constraints and Uncertainty". In: *2022 IEEE International Conference on Industrial Technology (ICIT)*. Aug. 2022, pp. 1–8. DOI: 10.1109/ICIT48603.2022.10002805. URL: https://ieeexplore.ieee.org/abstract/document/10002805 (visited on 10/14/2024).

[57] Eduardo Feo-Flushing, Luca Maria Gambardella, and Gianni A. Di Caro. "Spatially-Distributed Missions With Heterogeneous Multi-Robot Teams". In: *IEEE Access* 9 (2021). Conference Name: IEEE Access, pp. 67327–67348. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2021.3076919. URL: https://ieeexplore.ieee.org/abstract/document/9420064 (visited on 11/21/2024).

[58] Xinzhu Liu et al. “Embodied Multi-Agent Task Planning from Ambiguous Instruction”. en. In: *Robotics: Science and Systems XVIII*. Robotics: Science and Systems Foundation, June 2022. ISBN: 978-0-9923747-8-5. DOI: 10.15607/RSS.2022.XVIII.032. URL: http://www.roboticsproceedings.org/rss18/p032.pdf (visited on 12/12/2024).

[59] Mohammed Diab et al. “PMK—A Knowledge Processing Framework for Autonomous Robotics Perception and Manipulation”. en. In: *Sensors* 19.5 (Jan. 2019). Number: 5 Publisher: Multidisciplinary Digital Publishing Institute, p. 1166. ISSN: 1424-8220. DOI: 10.3390/s19051166. URL: https://www.mdpi.com/1424-8220/19/5/1166 (visited on 12/12/2024).

[60] Kehui Liu et al. *COHERENT: Collaboration of Heterogeneous Multi-Robot System with Large Language Models*. arXiv:2409.15146. Sept. 2024. DOI: 10.48550/arXiv.2409.15146. URL: http://arxiv.org/abs/2409.15146 (visited on 10/17/2024).

[61] Ziyuan Ma and Huajun Gong. “Heterogeneous multi-agent task allocation based on graph neural network ant colony optimization algorithms”. en. In: *Intelligence & Robotics* 3.4 (Oct. 2023), pp. 581–95. ISSN: 2770-3541, 2770-3541. DOI: 10.20517/ir.2023.33. URL: https://www.oaepublish.com/articles/ir.2023.33 (visited on 10/25/2024).

[62] Mitchell McIntire, Ernesto Nunes, and Maria Gini. “Iterated Multi-Robot Auctions for Precedence-Constrained Task Scheduling”. en. In: (May 2016).

[63] Luping Zhang and T. N. Wong. “An object-coding genetic algorithm for integrated process planning and scheduling”. In: *European Journal of Operational Research* 244.2 (July 2015), pp. 434–444. ISSN: 0377-2217. DOI: 10.1016/j.ejor.2015.01.032. URL: https://www.sciencedirect.com/science/article/pii/S0377221715000521 (visited on 11/21/2024).

[64] Zheyuan Wang and Matthew Gombolay. “Heterogeneous Graph Attention Networks for Scalable Multi-Robot Scheduling with Temporospatial Constraints”. en. In: (2020).

[65] Ishaq Ansari et al. “Cooperative and load-balancing auctions for heterogeneous multi-robot teams dealing with spatial and non-atomic tasks”. In: *2020 IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)*. ISSN: 2475-8426. Nov. 2020, pp. 213–220. DOI: 10.1109/SSRR50563.2020.9292605. URL: https://ieeexplore.ieee.org/abstract/document/9292605 (visited on 11/01/2024).

[66] Matthew Gombolay, Ronald Wilcox, and Julie Shah. “Fast Scheduling of Multi-Robot Teams with Temporospatial Constraints”. en. In: *Robotics: Science and Systems IX*. Robotics: Science and Systems Foundation, June 2013. ISBN: 978-981-07-3937-9. DOI: 10.15607/RSS.2013.IX.049. URL: http://www.roboticsproceedings.org/rss09/p49.pdf (visited on 11/26/2024).

[67] Saurabh Verma and Zhi-Li Zhang. *Graph Capsule Convolutional Neural Networks*. arXiv:1805.08090. Aug. 2018. DOI: 10.48550/arXiv.1805.08090. URL: http://arxiv.org/abs/1805.08090 (visited on 11/27/2024).

[68] Ronald J. Williams. “Simple statistical gradient-following algorithms for connectionist reinforcement learning”. en. In: *Machine Learning* 8.3 (May 1992), pp. 229–256. ISSN: 1573-0565. DOI: 10.1007/BF00992696. URL: https://doi.org/10.1007/BF00992696 (visited on 11/21/2024).

[69] Hakim Mitiche, Dalila Boughaci, and Maria Gini. “Iterated Local Search for Time-extended Multi-robot Task Allocation with Spatio-temporal and Capacity Constraints”. In: *Journal of Intelligent Systems* 28 (Dec. 2018). DOI: 10.1515/jisys-2018-0267.

[70] Pieter Vansteenwegen et al. “Iterated local search for the team orienteering problem with time windows”. In: *Computers & Operations Research*. New developments on hub location 36.12 (Dec. 2009), pp. 3281–3290. ISSN: 0305-0548. DOI: 10.1016/j.cor.2009.03.008. URL: https://www.sciencedirect.com/science/article/pii/S030505480900080X (visited on 11/27/2024).

[71] Payam Ghassemi, David DePauw, and Souma Chowdhury. “Decentralized Dynamic Task Allocation in Swarm Robotic Systems for Disaster Response: Extended Abstract”. In: *2019 International Symposium on Multi-Robot and Multi-Agent Systems (MRS)*. Aug. 2019, pp. 83–85. DOI: 10.1109/MRS.2019.8901062. URL: https://ieeexplore.ieee.org/document/8901062/?arnumber=8901062 (visited on 11/27/2024).

[72] Wouter Kool, Herke van Hoof, and Max Welling. *Attention, Learn to Solve Routing Problems!* arXiv:1803.08475. Feb. 2019. DOI: 10.48550/arXiv.1803.08475. URL: http://arxiv.org/abs/1803.08475 (visited on 11/27/2024).

[73] Ruisen Liu, Manisha Natarajan, and Matthew Gombolay. *Human-Robot Team Coordination with Dynamic and Latent Human Task Proficiencies: Scheduling with Learning Curves*. arXiv:2007.01921

[cs]. July 2020. DOI: 10.48550/arXiv.2007.01921. URL: http://arxiv.org/abs/2007.01921 (visited on 12/05/2024).

[74] Shaobo Zhang et al. “Real-Time Adaptive Assembly Scheduling in Human-Multi-Robot Collaboration According to Human Capability”. In: *2020 IEEE International Conference on Robotics and Automation (ICRA)*. ISSN: 2577-087X. May 2020, pp. 3860–3866. DOI: 10.1109/ICRA40945.2020.9196618. URL: https://ieeexplore.ieee.org/document/9196618 (visited on 10/16/2024).

[75] Yu Zhang and Lynne E. Parker. “Multi-robot task scheduling”. In: *2013 IEEE International Conference on Robotics and Automation*. ISSN: 1050-4729. May 2013, pp. 2992–2998. DOI: 10.1109/ICRA.2013.6630992. URL: https://ieeexplore.ieee.org/document/6630992/?arnumber=6630992 (visited on 11/25/2024).

[76] John E. Hunt and Denise E. Cooke. “Learning using an artificial immune system”. In: *Journal of Network and Computer Applications* 19.2 (Apr. 1996), pp. 189–212. ISSN: 1084-8045. DOI: 10.1006/jnca.1996.0014. URL: https://www.sciencedirect.com/science/article/pii/S1084804596900144 (visited on 11/15/2024).

[77] Amanda Prorok. “Redundant Robot Assignment on Graphs with Uncertain Edge Costs”. en. In: *Distributed Autonomous Robotic Systems*. Ed. by Nikolaus Correll, Mac Schwager, and Michael Otte. Cham: Springer International Publishing, 2019, pp. 313–327. ISBN: 978-3-030-05816-6. DOI: 10.1007/978-3-030-05816-6_22.

[78] Álvaro Calvo and Jesús Capitán. “Optimal Task Allocation for Heterogeneous Multi-robot Teams with Battery Constraints”. In: *2024 IEEE International Conference on Robotics and Automation (ICRA)*. 2024, pp. 7243–7249. DOI: 10.1109/icra57147.2024.10611147.

[79] Anirudha Majumdar and Marco Pavone. “How Should a Robot Assess Risk? Towards an Axiomatic Theory of Risk in Robotics”. en. In: *Robotics Research*. Ed. by Nancy M. Amato et al. Vol. 10. Series Title: Springer Proceedings in Advanced Robotics. Cham: Springer International Publishing, 2020, pp. 75–84. ISBN: 978-3-030-28618-7 978-3-030-28619-4. DOI: 10.1007/978-3-030-28619-4_10. URL: http://link.springer.com/10.1007/978-3-030-28619-4_10 (visited on 12/06/2024).

[80] Dian Rachmawati and Lysander Gustin. “Analysis of Dijkstra’s Algorithm and A* Algorithm in Shortest Path Problem”. en. In: *Journal of Physics: Conference Series* 1566.1 (June 2020), p. 012061. ISSN: 1742-6588, 1742-6596. DOI: 10.1088/1742-6596/1566/1/012061. URL: https://iopscience.iop.org/article/10.1088/1742-6596/1566/1/012061 (visited on 12/06/2024).

[81] Bo Fu et al. “Heterogeneous vehicle routing and teaming with gaussian distributed energy uncertainty”. In: *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2020, pp. 4315–4322. URL: https://ieeexplore.ieee.org/abstract/document/9341433/ (visited on 11/04/2024).

[82] M W P Savelsbergh and M Goetschalckx. “An Efficient Approximation Algorithm for the Fixed Routes Problem”. en. In: (1992).

[83] Amanda Prorok et al. *The Holy Grail of Multi-Robot Planning: Learning to Generate Online-Scalable Solutions from Offline-Optimal Experts*. arXiv:2107.12254. July 2021. URL: http://arxiv.org/abs/2107.12254 (visited on 10/15/2024).

[84] Ayan Dutta et al. “Distributed Hedonic Coalition Formation for Multi-Robot Task Allocation”. In: *2021 IEEE 17th International Conference on Automation Science and Engineering (CASE)*. ISSN: 2161-8089. Aug. 2021, pp. 639–644. DOI: 10.1109/CASE49439.2021.9551582. URL: https://ieeexplore.ieee.org/document/9551582 (visited on 10/17/2024).

[85] Marco Dorigo and Thomas Stützle. “Ant Colony Optimization: Overview and Recent Advances”. en. In: *Handbook of Metaheuristics*. Ed. by Michel Gendreau and Jean-Yves Potvin. Cham: Springer International Publishing, 2019, pp. 311–351. ISBN: 978-3-319-91086-4. DOI: 10.1007/978-3-319-91086-4_10. URL: https://doi.org/10.1007/978-3-319-91086-4_10 (visited on 11/06/2024).

[86] Ahmed G. Gad. “Particle Swarm Optimization Algorithm and Its Applications: A Systematic Review”. en. In: *Archives of Computational Methods in Engineering* 29.5 (Aug. 2022), pp. 2531–2561. ISSN: 1886-1784. DOI: 10.1007/s11831-021-09694-4. URL: https://doi.org/10.1007/s11831-021-09694-4 (visited on 12/03/2024).

[87] Liangjun Ke, Qingfu Zhang, and Roberto Battiti. “MOEA/D-ACO: A Multiobjective Evolutionary Algorithm Using Decomposition and AntColony”. In: *IEEE Transactions on Cybernetics* 43.6 (Dec. 2013). Conference Name: IEEE Transactions on Cybernetics, pp. 1845–1859. ISSN: 2168-2275.

DOI: 10.1109/TSMCB.2012.2231860. URL: https://ieeexplore.ieee.org/document/6423879 (visited on 12/04/2024).

[88] Xue Yu et al. “Set-Based Discrete Particle Swarm Optimization Based on Decomposition for Permutation-Based Multiobjective Combinatorial Optimization Problems”. In: *IEEE Transactions on Cybernetics* 48.7 (July 2018). Conference Name: IEEE Transactions on Cybernetics, pp. 2139–2153. ISSN: 2168-2275. DOI: 10.1109/TCYB.2017.2728120. URL: https://ieeexplore.ieee.org/document/8003382/?arnumber=8003382 (visited on 12/04/2024).

[89] Mincan Li et al. “Task Allocation on Layered Multiagent Systems: When Evolutionary Many-Objective Optimization Meets Deep Q-Learning”. In: *IEEE Transactions on Evolutionary Computation* 25.5 (Oct. 2021). Conference Name: IEEE Transactions on Evolutionary Computation, pp. 842–855. ISSN: 1941-0026. DOI: 10.1109/TEVC.2021.3049131. URL: https://ieeexplore.ieee.org/document/9313054 (visited on 12/04/2024).

[90] Athira K A, Divya Udayan J, and Umashankar Subramaniam. “A Systematic Literature Review on Multi-Robot Task Allocation”. In: *ACM Comput. Surv.* 57.3 (Nov. 2024), 68:1–68:28. ISSN: 0360-0300. DOI: 10.1145/3700591. URL: https://dl.acm.org/doi/10.1145/3700591 (visited on 12/04/2024).

[91] Jiageng Mao et al. *3D Object Detection for Autonomous Driving: A Comprehensive Survey*. arXiv:2206.09474 [cs]. Apr. 2023. DOI: 10.48550/arXiv.2206.09474. URL: http://arxiv.org/abs/2206.09474 (visited on 12/04/2024).

[92] Rui Qian, Xin Lai, and Xirong Li. “3D Object Detection for Autonomous Driving: A Survey”. In: *Pattern Recognition* 130 (Oct. 2022), p. 108796. ISSN: 0031-3203. DOI: 10.1016/j.patcog.2022.108796. URL: https://www.sciencedirect.com/science/article/pii/S0031320322002771 (visited on 12/04/2024).

[93] Giacomo Da Col and Erich Teppan. *Large-Scale Benchmarks for the Job Shop Scheduling Problem*. en. arXiv:2102.08778 [cs]. Mar. 2021. DOI: 10.48550/arXiv.2102.08778. URL: http://arxiv.org/abs/2102.08778 (visited on 12/04/2024).

[94] Aldy Gunawan et al. “Vehicle routing: Review of benchmark datasets”. In: *Journal of the Operational Research Society* 72.8 (Aug. 2021). Publisher: Taylor & Francis _eprint: https://doi.org/10.1080/01605682.20 pp. 1794–1807. ISSN: 0160-5682. DOI: 10.1080/01605682.2021.1884505. URL: https://doi.org/10.1080/01605682.2021.1884505 (visited on 12/04/2024).

[95] Behice Meltem Kayhan and Gokalp Yildiz. “Reinforcement learning applications to machine scheduling problems: a comprehensive literature review”. en. In: *Journal of Intelligent Manufacturing* 34.3 (2023). Publisher: Springer, pp. 905–929. URL: https://ideas.repec.org//a/spr/joinma/v34y2023i3d10.1007_s10845-021-01847-3.html (visited on 10/22/2024).

[96] Bumjin Park, Cheongwoong Kang, and Jaesik Choi. “Cooperative Multi-Robot Task Allocation with Reinforcement Learning”. en. In: *Applied Sciences* 12.1 (Jan. 2022). Number: 1 Publisher: Multidisciplinary Digital Publishing Institute, p. 272. ISSN: 2076-3417. DOI: 10.3390/app12010272. URL: https://www.mdpi.com/2076-3417/12/1/272 (visited on 10/14/2024).